

UECS ノード開発用ミドルウェア UARDECS_PICO Ver1.0.x

説明書

1.0 版

Written by Hideto Kurosaki

2026. 6. 29.

1 はじめに

UARDECS（ユアルデックス）は開発環境に無償で入手可能な Arduino IDE を用い、安価な Raspberry Pi Pico または Pico2 およびその互換機を利用して [ユビキタス環境制御システム](#) (UECS) のノードを作成するために開発されたミドルウェアであり、必要最低限の機能を搭載したノードの開発を容易なものとし、施設園芸の環境制御システムの構築を今まで以上に簡単にすることを目的としています。このミドルウェアは、[UECS の通信実用規約”1.00-E10”](#) に対応した通信文の送受信管理とマイコンボードに搭載した http サーバを利用した設定画面の生成、設定値の不揮発性メモリへの自動読み書き等を行うことができます。



UARDECS_PICO で作成した気象観測ノード

UARDECS は以下のサイトから提供されています

GitHub リポジトリ

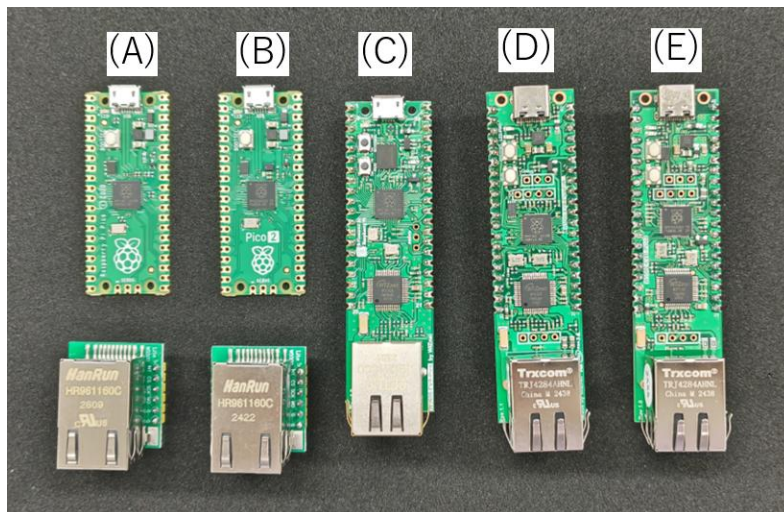
https://github.com/H-Kurosaki/UARDECS_PICO

制限事項

○本ソフトウェアの利用は、同封のライセンス条項に同意した上で行ってください。

2 対応機種

UARDECS_PICO はターゲットとするマイコンボードを以下の機種の組み合わせとしています。さらに、最低限の開発環境を構築するには、これらのマイコンボードに加えて、開発用 PC、USB ケーブル、LAN ケーブル、電源用 AC アダプタを準備する必要があります。



(A) Raspberry Pi Pico^{※1} + W5500 LAN モジュール

(B) Raspberry Pi Pico2 + W5500 LAN モジュール

(C) WIZnet W5500-EVB-Pico^{※1}

(D) WIZnet W5500-EVB-Pico-PoE^{※1※2}

(E) WIZnet W5500-EVB-Pico2 (推奨)

これ以外にも RP2040 および RP2350 搭載機に適切な結線を行えば動作する可能性があります。本ライブラリは W5500 というイーサネットコントローラー IC に対応しています。Raspberry Pi Pico 単体では有線 LAN 端子をもたないため、別途増設する必要があります。WIZnet 社より W5500-EVB-Pico または W5500-EVB-Pico2 という互換機が販売されています。これらの機種は最初から有線 LAN ポートを持ち W5500 が結線された状態になっているため、開発が容易にできます。

※1 Raspberry Pi Pico およびその互換機 (RP2040 搭載機) ではエラッタにより AD コンバータの精度が低いという問題があり、アナログ入力を用いて電圧を測定する用途には Raspberry Pi Pico2 互換機 (RP2350 搭載機) を推奨します。

※2 W5500-EVB-Pico-PoE と W5500-EVB-Pico2 は外観がよく似ているので注意すること (コンパイル時に設定を間違えるとプログラムが動作しない)

3 他の UARDECS との違い

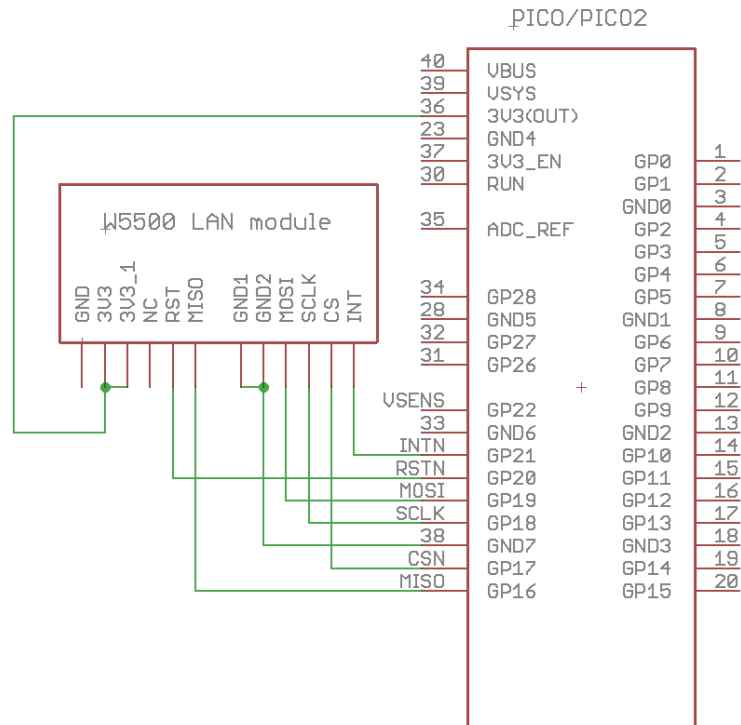
これまで開発された Arduino 用ライブラリ “UARDECS” , “UARDECS_MEGA” と “UARDECS_PICO” の比較を示します. UARDECS_PICO は UARDECS_MEGA をベースに開発されており, 仕様には共通点があります.

	UARDECS	UARDECS_MEGA	UARDECS_PICO
主な対応機種	Arduino UNO Arduino MEGA	Arduino MEGA	Raspberry Pi PICO Raspberry Pi PICO2
消費メモリ	少	多	多
CCM の Type 文字列 の書き換え	ソースコード上 にハードコーディング	Web 上で編集可能 ^{※1}	Web 上で編集可能 ^{※1}
CCM の room- region-order の扱い	Web 上でノード 単位に設定	Web 上で CCM ごとに 別々に設定可能	Web 上で CCM ごとに 別々に設定可能
設定可能な CCM 送 信レベル	A_1S_0 A_10S_0 A_1M_0 S_1S_0	A_1S_0 A_10S_0 A_10S_1 ^{※2} A_1M_0 A_1M_1 ^{※2} S_1S_0	A_1S_0 A_10S_0 A_10S_1 ^{※2} A_1M_0 A_1M_1 ^{※2} S_1S_0
入力可能な URL 文 字列長	300byte	500byte	500byte
EEPROM の利用範囲	0x320 以降	0x9AC 以降	プログラム用フラッシュメモ リの一部を流用 (メモリ配置は MEGA と同じ)
その他		UARDECS 用のソースコ ードをそのままコンパ イル可能	ハードウェア依存しない 部分では UARDECS_MEGA のコードと高い互換性が ある.

※1 プログラム上に記述した Type 文字列はデフォルト値という解釈になる

※2 値が変化した場合は 1 秒以内に送信される. 最大送信頻度は 1 秒間隔を超えることはない. A_1S_1 は実装できない.

4 Raspberry Pi Pico の結線について



Raspberry Pi Pico で W5500-EVB-Pico と同等の機能を得る結線方法 (Pico2 でも同様)

I/O	Pin Name	Description
I	GPIO16	Connected to MISO on W5500
O	GPIO17	Connected to CSn on W5500
O	GPIO18	Connected to SCLK on W5500
O	GPIO19	Connected to MOSI on W5500
O	GPIO20	Connected to RSTn on W5500
I	GPIO21	Connected to INTn on W5500
I	GPIO24	VBUS sense - high if VBUS is present, else low
O	GPIO25	Connected to user LED
I	GPIO29	Used in ADC mode (ADC3) to measure VSY/3

W5500 モジュールの結線により使用されるピン一覧 (W5500-EVB-Pico の場合)
 (<https://docs.wiznet.io/Product/Chip/Ethernet/W5500/w5500-evb-pico> より引用)

- 1) 有線 LAN 端子を持たない Raspberry Pi Pico および Raspberry Pi Pico2 を用いる場合、別途販売されている W5500 LAN モジュール (W5500 イーサネットモジュールなど) を図のように結線する必要があります。
- 2) W5500-EVB-Pico, W5500-EVB-Pico2 など、最初から LAN 端子が実装されている機種を用いる場合、特に追加の回路なしに UARDECS_PICO を実行可能です。ただし、最初から W5500 が結線されているため、複数のピンが占有されています。GPIO16~21, 24, 25, 29 です。このうち GPIO 16, 18, 19 は SPI 通信の MISO, SCLK, MOSI にあたるため、別途 CS ピンを消費して SPI 通信可能なデバイスに限り同じピンを共有して接続できます。
- 3) W5500 の追加により 3.3V の電源を 170mA 程度消費することがあります。Raspberry Pi Pico の 3.3V の余剰出力は 300mA ほどあるので、そのままでも動作は可能ですが、電力を多く消費するデバイスを追加すると電力不足に陥ることがあります。その場合、電源を別に取りする必要があります。
- 4) W5500 の入出力ピンは 3.3V~5V トレラントであり、バッファがあるためマイコンと異なる電源から駆動してもラッチアップの危険性は少ないのですが、これ以外のデバイスを Raspberry Pi Pico 内蔵の電源以外で駆動して接続する場合、電圧差によるラッチアップが発生する危険性があるため、そのようなトラブルを回避可能な回路構成にしてください。

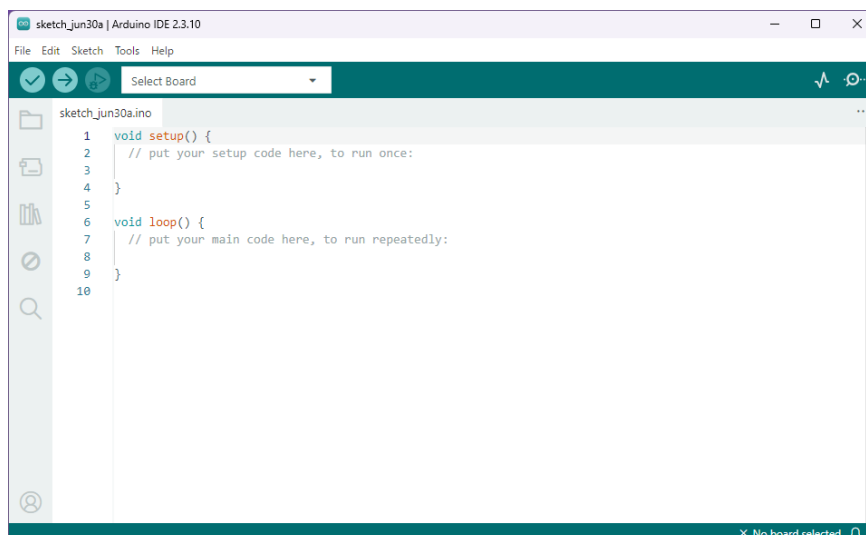
5 開発環境の構築

Windows PC を利用して開発環境を構築する例を説明します。UARDECS_PICO を使用するには開発用 PC に Arduino IDE をインストールする必要があります。旧バージョンの Ver1.8.19 と新バージョンの Ver2.x がありますがどちらも動作します（ダウンロード先：<https://www.arduino.cc/>）

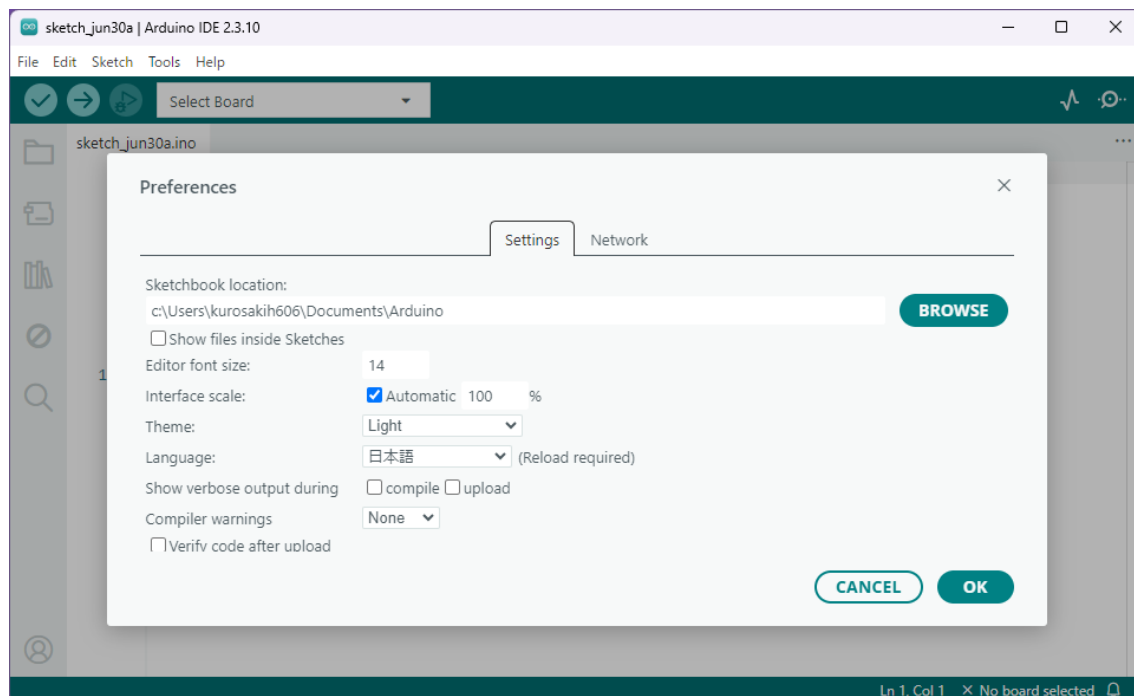
- 1) Arduino の開発環境をダウンロードします。インストーラ付きの Windows 用を使用します。本手順では参考として 2.x 以降の手順を説明します。これを開発用 PC にインストールします。以降の処理はすべてネットに接続して行ってください。



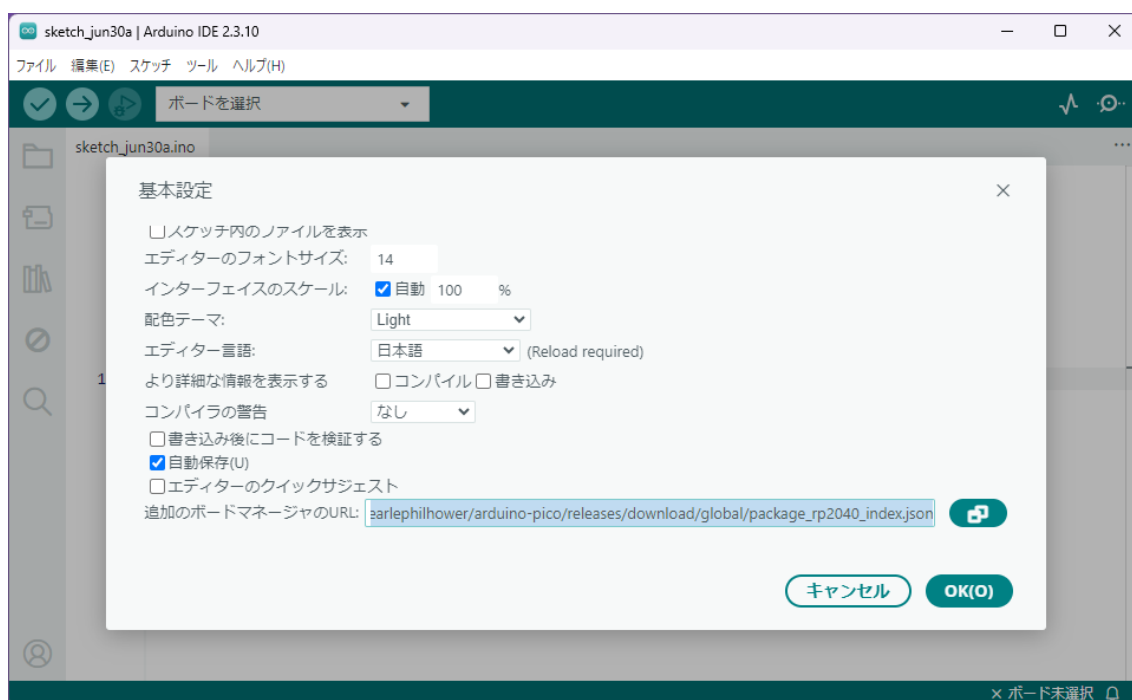
- 2) Arduino IDE を起動すると（管理者アカウントではない場合、ネットワークアクセスの許可が複数回求められますがすべて許可してください）以下の画面が出てきます。



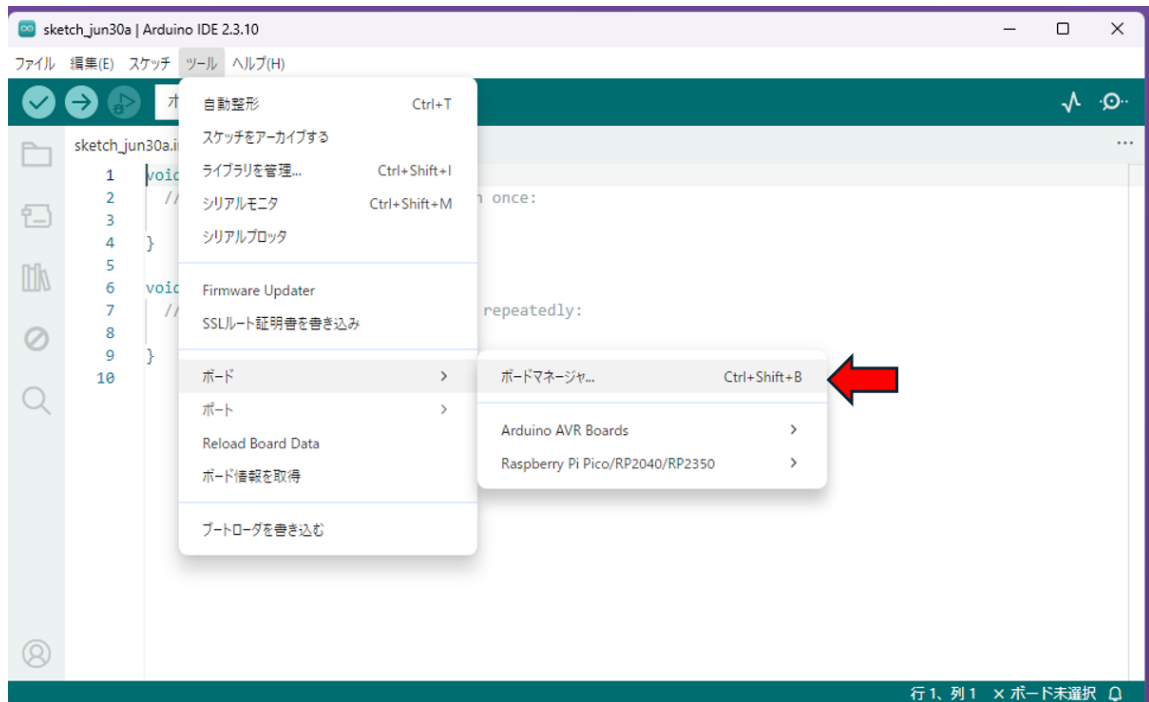
- 3) 標準では英語表示なので、日本語で使用したい場合 File→Preferences に入り、Language に日本語を設定して OK ボタンを押すと表示を日本語化できます。



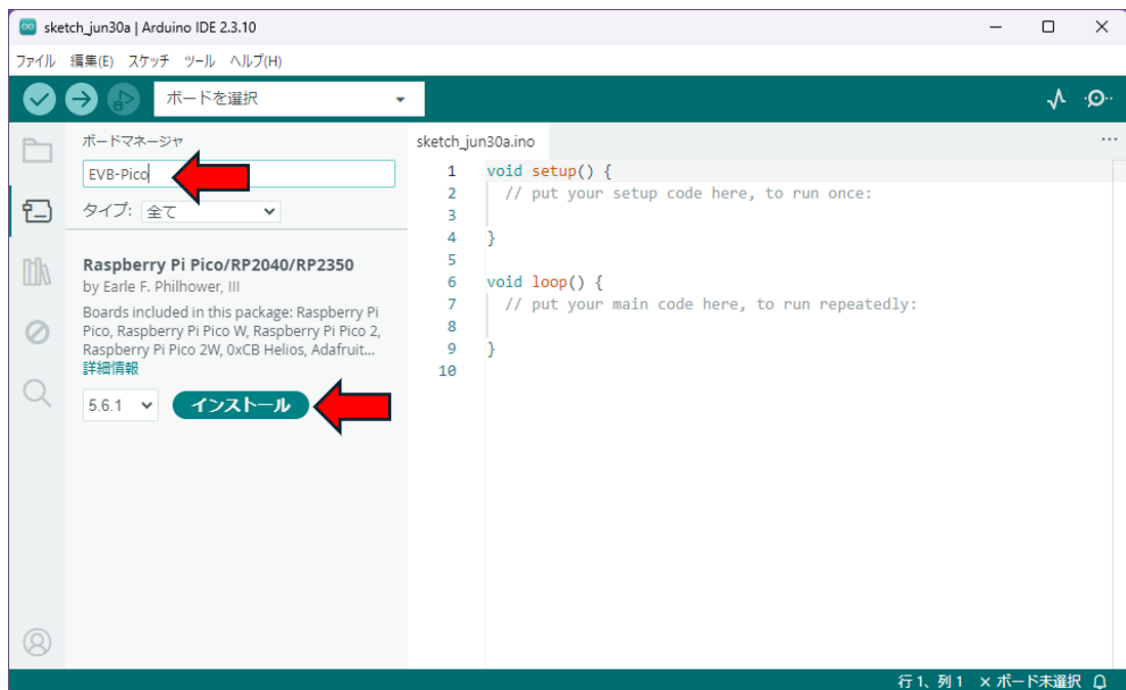
- 4) さらにファイル→基本設定 (File→Preferences) を下の方にスクロールすると追加ボードマネージャの URL という項目があるのでここに
「https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json」 と入力して OK ボタンをクリックします。



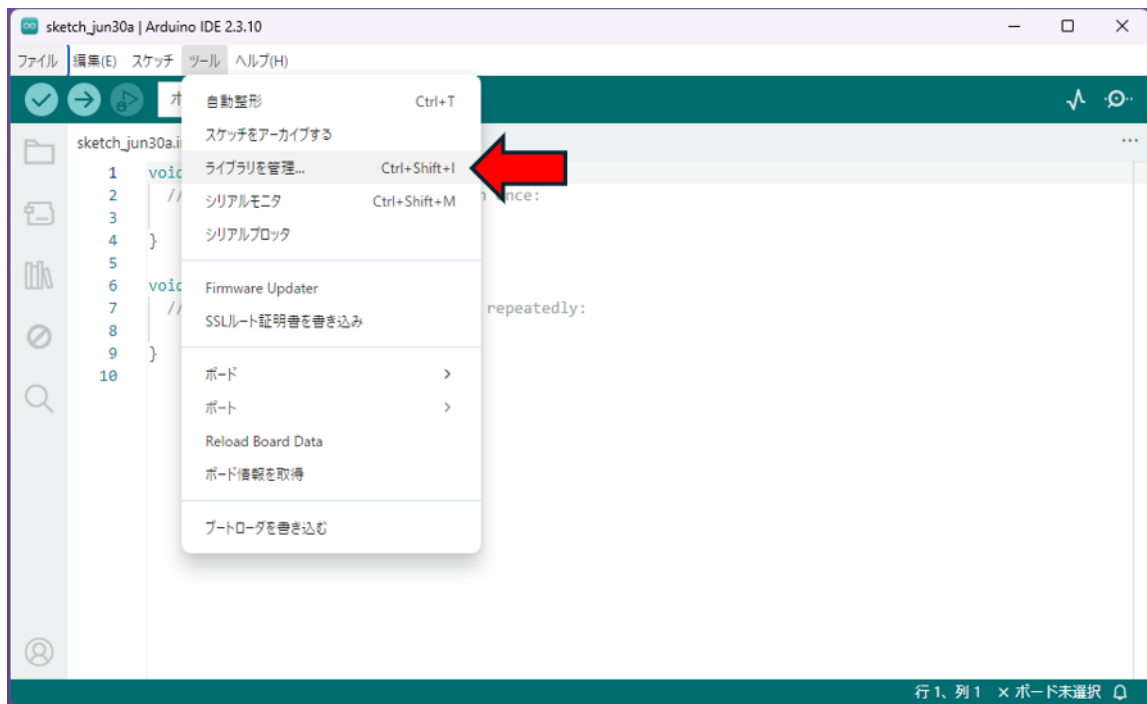
5) ツール→ボード→ボードマネージャに入ります



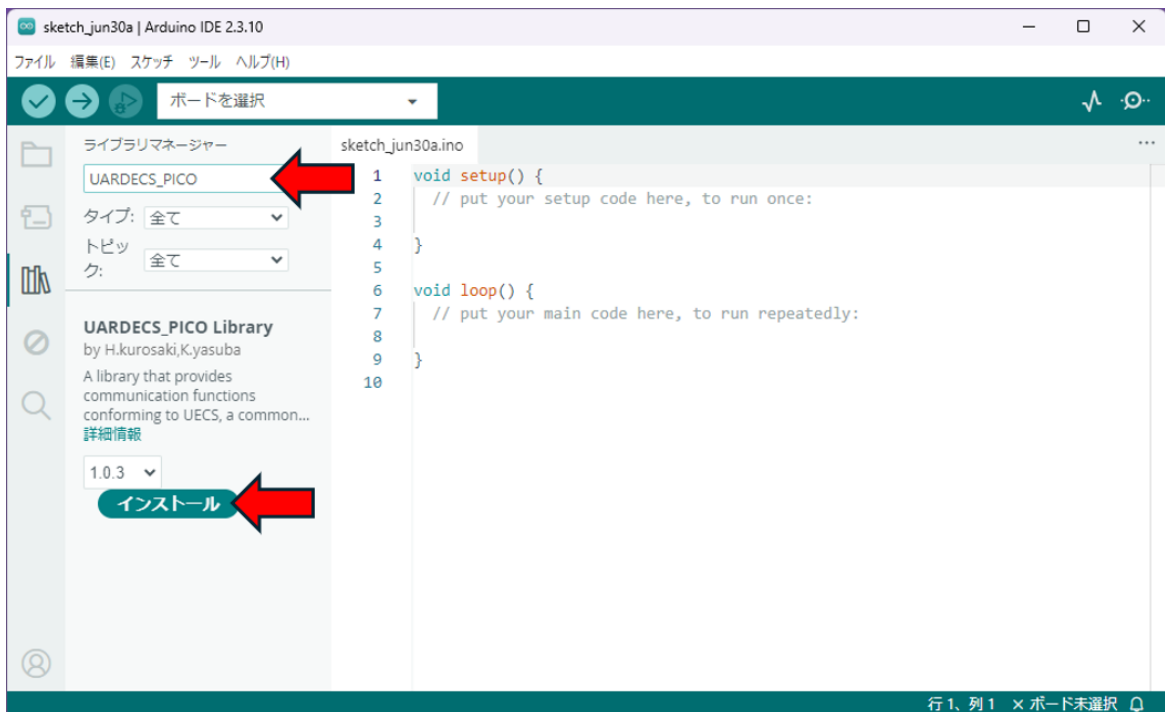
6) ボードマネージャの検索欄に” EVB-Pico” と入力すると出てくるボードがあるのでインストールします。



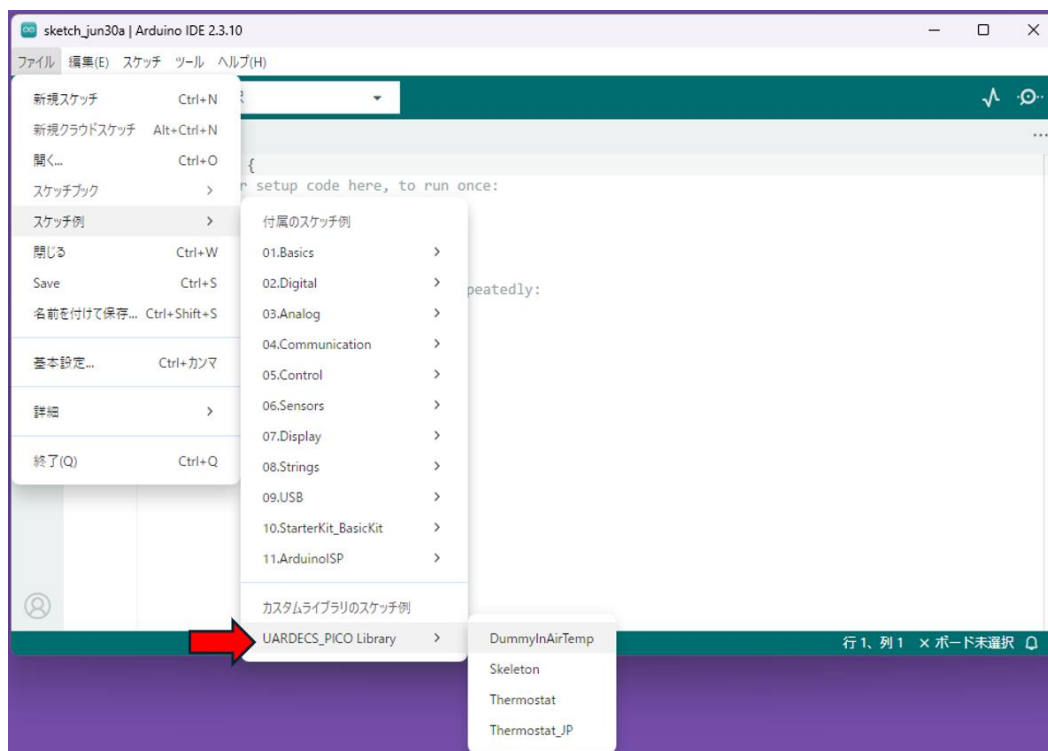
7) ツール→ライブラリを管理に入ります



8) 検索欄に”UARDECS_PICO” と入力すると出てくるものがあるのでインストールします



- 9) 正常に UARDECS_PICO がインストールされている場合、Arduino IDE を起動したとき図の位置に UARDECS のサンプルプログラムのリストが表示されます。(ほかのライブラリがインストールされている場合、もっと下にあるかもしれません)



- 10) 以上で開発環境の構築は終わりです。一度 Arduino IDE を終了してください。以降、この PC にこれらの作業をする必要はありません。

6 マイコンの準備と開発用 PC の IP アドレス設定

- 1) マイコンに USB ケーブルと LAN ケーブル(普通のストレートケーブルで良い)を接続し、両方とも開発用 PC に接続します。正常に認識されればシリアル通信用のドライバがインストールされます。(この状態ではマイコンは PC の USB ポートから給電されて動作します)
- 2) 開発用 PC のマイコンに接続した LAN ポートの IP アドレスを以下の値に設定し直します。

IP アドレス : 192. 168. 1. 1

サブネットマスク : 255. 255. 255. 0

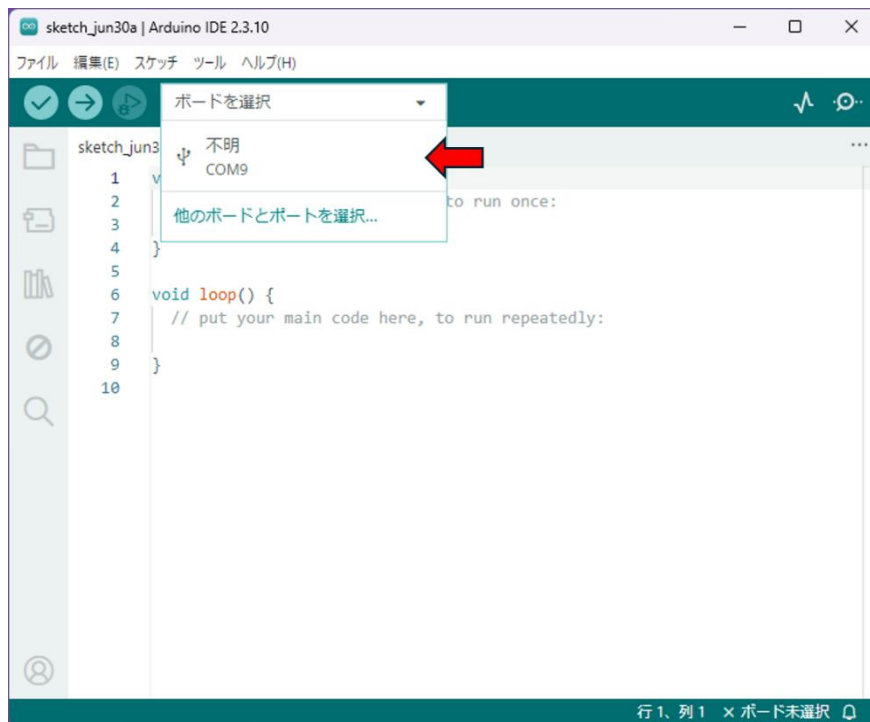
デフォルトゲートウェイ : 変更不要

DNS サーバーアドレス : 変更不要

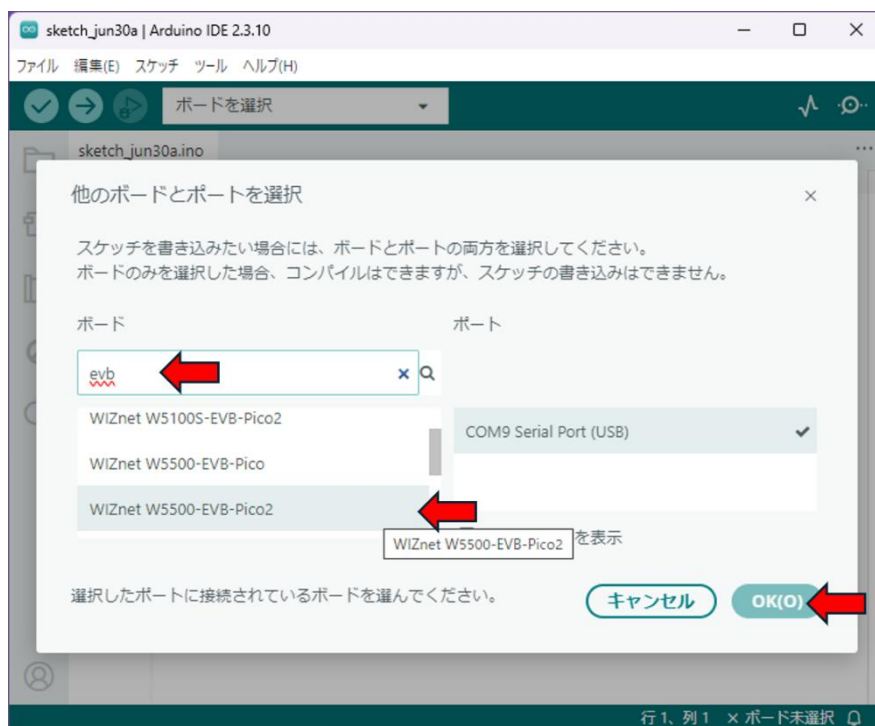
注意 : 設定を変更するとインターネットに接続できなくなることがあります。不安な方は後で設定を戻せるように変更前の状態をメモして下さい。

注意 : 複数のイーサネットポート (LAN ポートが 2 つある, LAN ポートと無線 LAN 機能があるなど) がある PC では UDP パケットのブロードキャスト時に混乱を招くことがあります。通信に問題が生じる場合, 不要な LAN ポートのケーブルを抜く, 接続を解除するなどして無効にして下さい。

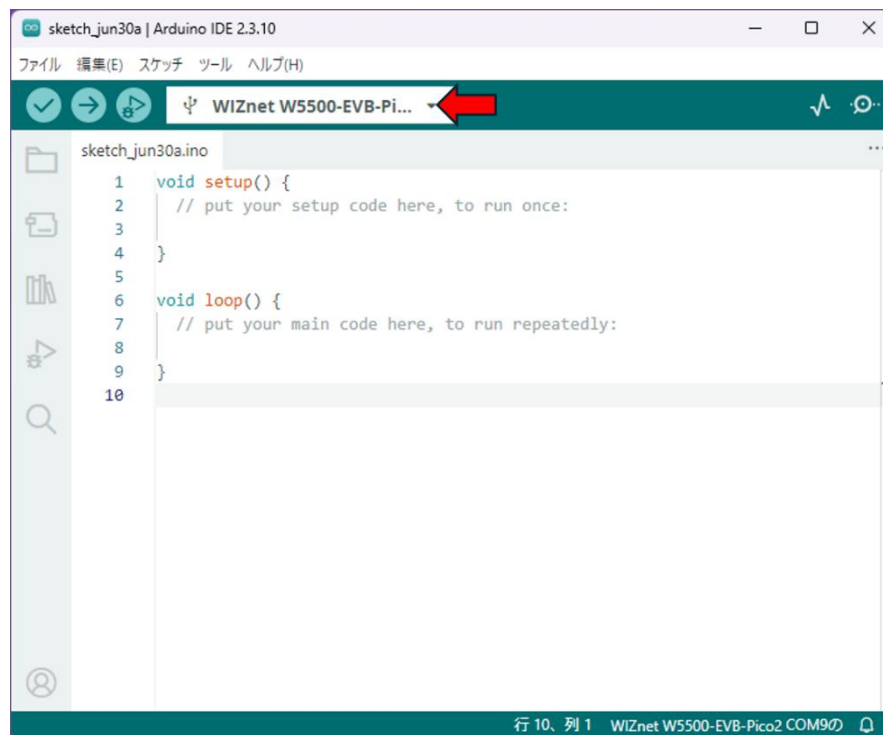
- 3) Arduino IDE を起動し「ボードを選択」の欄をクリックします。接続したマイコンが認識されている場合「不明 COM??」と表示されるのでここをクリック (COM??の数字は変わる可能性があります)



- 4) 検索欄に使用している機種名の一部を入れると候補が表示されるので、その中から一致するものを選んで OK ボタンをクリックします(図では W5500-EVB-Pico2).

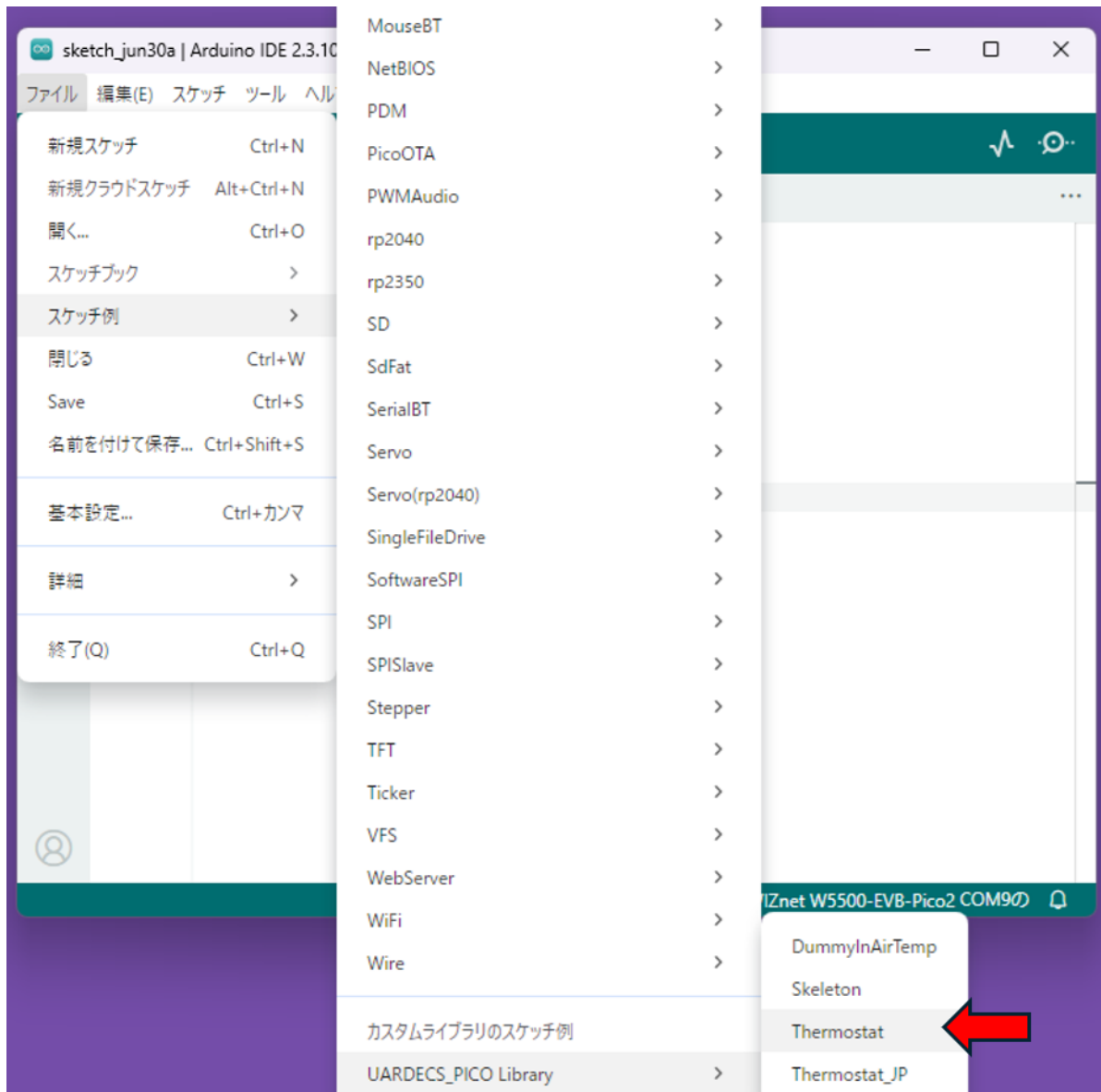


5) 上部に機種名が表示されていれば問題ないです.

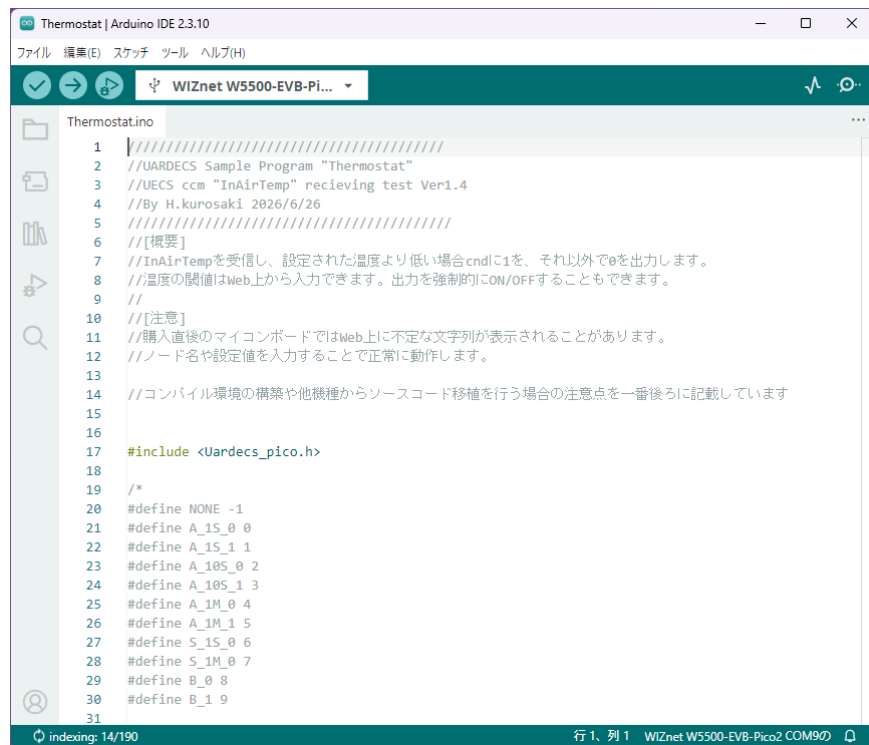


7 サンプルプログラムの実行

- 1) ファイル→スケッチの例→UARDECS_PICO Libraryの中から試しに Thermostat を呼び出してみます。かなり下の方にあるのでスクロールして選んでください。

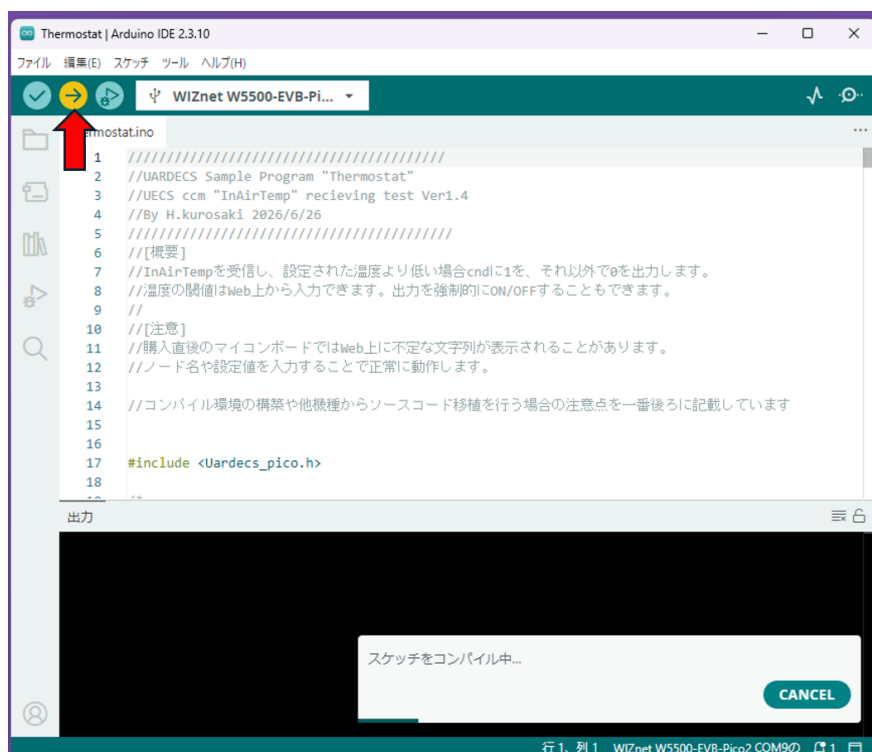


- 2) サンプルが読み込まれ新しいウィンドウが開きます。古いウィンドウは閉じてください。

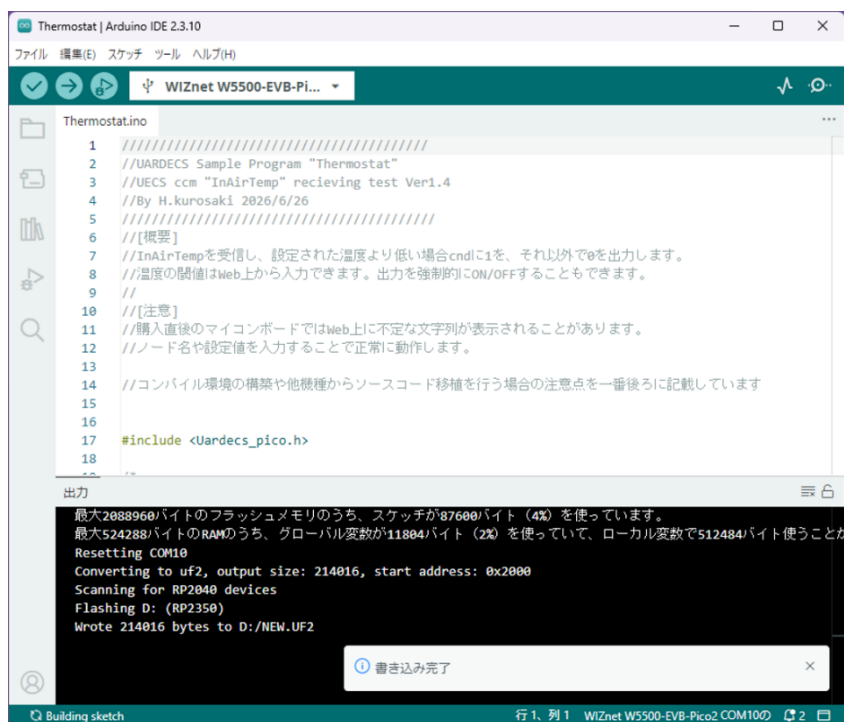


```
1 ///////////////////////////////////////////////////////////////////
2 //UARDECS Sample Program "Thermostat"
3 //UECS ccm "InAirTemp" recieving test Ver1.4
4 //By H.kurosaki 2026/6/26
5 ///////////////////////////////////////////////////////////////////
6 //[概要]
7 //InAirTempを受信し、設定された温度より低い場合cndlに1を、それ以外で0を出力します。
8 //温度の閾値はWeb上から入力できます。出力を強制的にON/OFFすることもできます。
9 //
10 //[注意]
11 //購入直後のマイコンボードではWeb上に不定な文字列が表示されることがあります。
12 //ノード名や設定値を入力することで正常に動作します。
13
14 //コンパイル環境の構築や他機種からソースコード移植を行う場合の注意点を一番後ろに記載しています
15
16
17 #include <Uardecs_pico.h>
18
19 /*
20 #define NONE -1
21 #define A_1S_0 0
22 #define A_1S_1 1
23 #define A_10S_0 2
24 #define A_10S_1 3
25 #define A_1M_0 4
26 #define A_1M_1 5
27 #define S_1S_0 6
28 #define S_1M_0 7
29 #define B_0 8
30 #define B_1 9
31
```

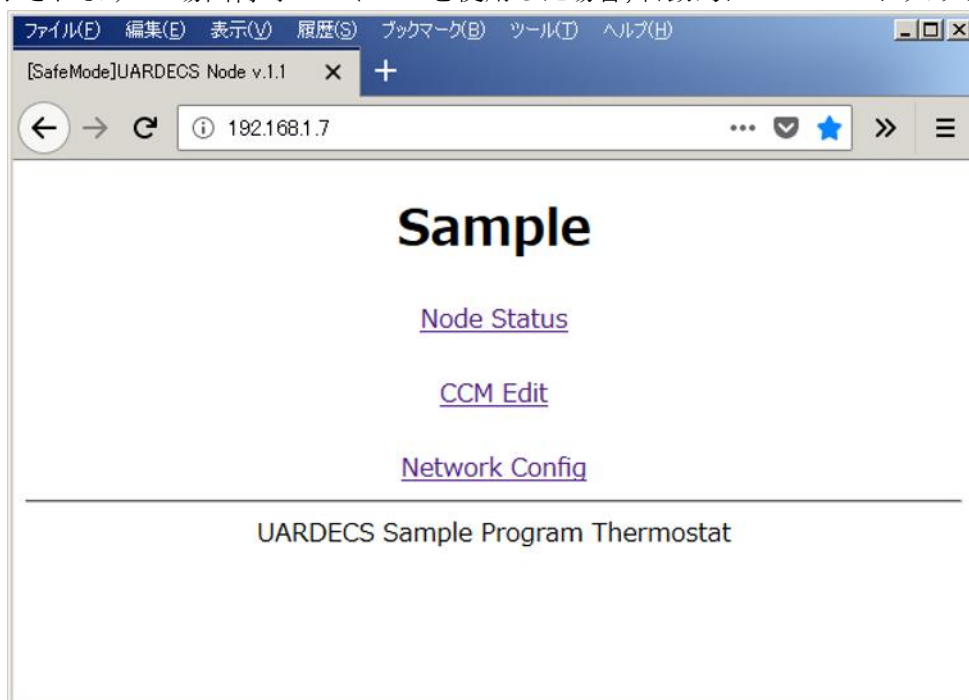
- 3) 上の→ボタンをクリックするとコンパイルが開始され自動的にプログラムがマイコンに書き込まれます。



- 4) 一瞬別のウィンドウが出ますが、閉じて構いません、成功すると書き込み完了と表示されます。



- 5) プログラムを書き込んだノードの動作を確認します。Web ブラウザを用いて” 192.168.1.7” にアクセスすると、ノードが正常に動作している場合、下の画面が表示されます。工場出荷時のマイコンを使用した場合、自動的に SafeMode に入ります。



SafeMode について：

工場出荷時の IP アドレスが未設定な状態か，設定した IP アドレスが分からなくなった時に，SafeMode ジャンパーを設定して起動すると SafeMode に入ります．この状態では Web 画面のタイトルの部分に[SafeMode]と表示されます．

SafeMode は時にはノードの設定にかかわらず，以下の IP アドレスが設定されます

IP アドレス : 192.168.1.7

サブネットマスク : 255.255.255.0

SafeMode ジャンパーについて：

Thermostat のサンプルスケッチ中に

```
const byte U_InitPin = 3;
```

と書かれているのが SafeMode ジャンパー用に使用するマイコンのピンです．最初に GPIO3 が指定されていますが，別のピンにも変更可能です※1．

```
const byte U_InitPin_Sense=HIGH;
```

と書かれているのが検出条件で，SafeMode ジャンパーがこの値を示した時に SafeMode に入ります．SafeMode ジャンパーは自動プルアップされる(すなわち HIGH になる)ので，Thermostat のサンプルスケッチは GPIO3 に何も接続しない場合，強制的に SafeMode で動作することを示しています．SafeMode を抜きたい場合，GPIO3 と GND 間をつなぐか，U_InitPin_Sense=LOW;に書き換えて下さい．

※1 GPIO3 は I²C 通信に使用することがあるので，衝突を回避する場合は別の未使用のピンを使用してください．

- 6) トップページの Network Config にアクセスすると IP アドレスを設定できます。工場出荷時のマイコンではすべての値が 255 になっています。IP アドレスの付け方については、ネット上に多数の文献がありますので参考にして下さい。図では IP アドレスに 192.168.1.7、サブネットマスクに 255.255.255.0 を設定しています。IP アドレス等の設定が書き換わると、リセットを促すメッセージが表示され、リセット後に設定が有効になります。ただし、SafeMode を抜けない限り、IP アドレス設定が有効になることはありません。ノード名には、英数字で 19 文字、日本語 6 文字以内の文字が設定できます。この画面での設定内容は不揮発性メモリに自動的に記録され、電源を切っても値が消えることはありません。デフォルトゲートウェイと DNS サーバも設定可能ですが、UARDECS は現在のバージョンではこの値を活用していません。

Sample

LAN

要設定 { address: 192 : 168 : 1 : 7
 subnet: 255 : 255 : 255 : 0

未使用 { gateway: 255 : 255 : 255 : 255
 dns: 255 : 255 : 255 : 255

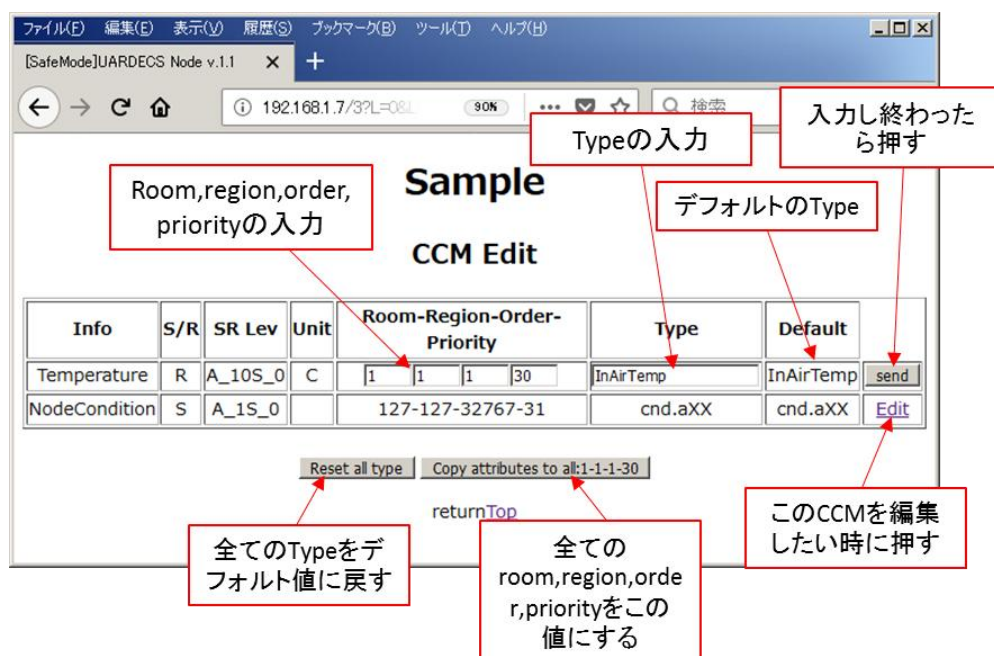
mac: 000000000000
uecsid: 000000000000

Node Name

Sample
send

[returnTop](#)

7) 次に CCM Edit に入ります。すると、下のような画面が表示されます。UARDECS_PICO では全ての CCM の room, region, order, priority をユーザーが CCM ごとにバラバラに設定することができます。また、Type の文字列も編集できます。



値の修正を行ったら必ず send ボタンを押さないと反映されません。編集する CCM を変更する場合は Edit を押します。下の方にある 2 つのボタンは、全ての type をデフォルトに戻すボタンと、現在編集中の room, region, order, priority の値を他の全ての CCM にコピーするボタンです。UARDECS_PICO では新しいプログラムをマイコンに転送したときに Type に不定な文字列が表示されることがあります。その場合、全ての Type をデフォルトに戻すボタンを押して修正してください。

- 8) トップページから Node Status に入るとサーモスタットの動作を設定できます。試しに UserSwitch に AUTO を、SetTemp に 10 を指定して send ボタンを押すと、設定が書き込まれます。ここで設定した値は不揮発性メモリに書き込まれ、ノードの電源を切っても設定値が残ります。ただし、SafeMode では電源投入後に値の復帰は行われず、ソースコード上に記述した初期値が入力されます(SafeMode 中でも値は保存されています)。工場出荷時のマイコンを使用している場合、あるいは他の用途に使用していたマイコンでは、プログラムの許容外の値がフラッシュメモリに書かれている可能性があるため、必ず 1 回 send ボタンをクリックしてください。この操作によりフラッシュメモリ上の異常値を消去することができます。

Sample
CCM Status

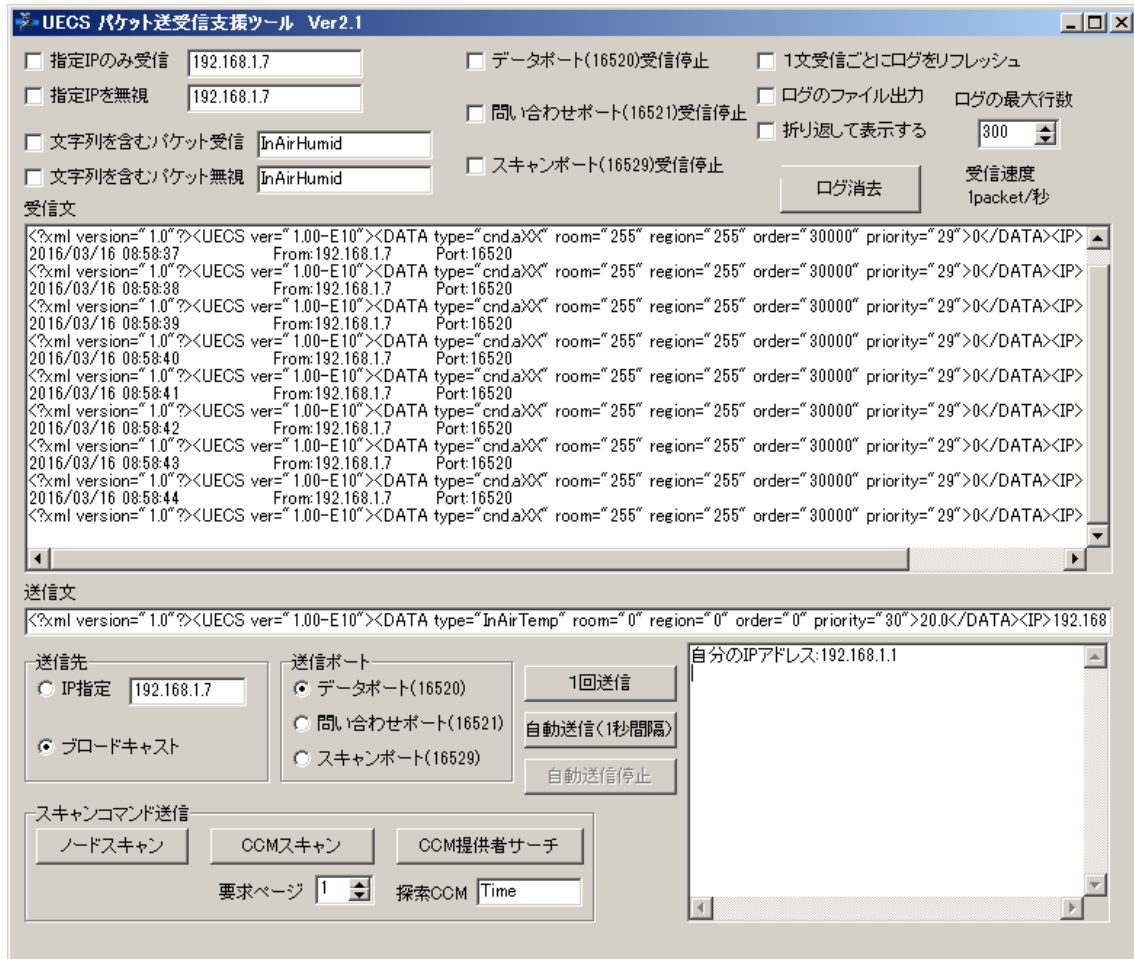
Info	S/R	Type	SR Lev	Value	Valid	Sec	Atr	IP
Temperature	R	InAirTemp	A_10S_0	0.0	-		(1-2-3)	255.255.255.255
NodeCondition	S	cnd.aXX	A_1S_0	0			(1-2-3)	255.255.255.255

Status & SetValue

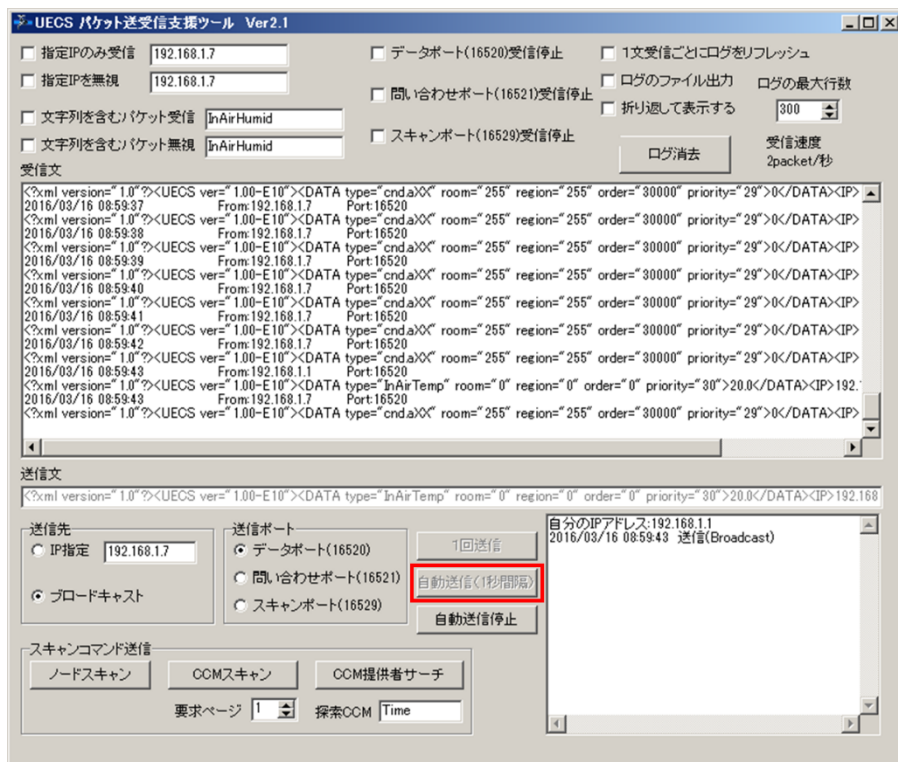
Name	Val	Unit	Detail
Temperature	0.0	C	SHOWDATA
UserSwitch	AUTO		SELECTDATA
SetTemp	10.0	C	INPUTDATA
Now status	OUTPUT:OFF		SHOWSTRING

[returnTop](#)

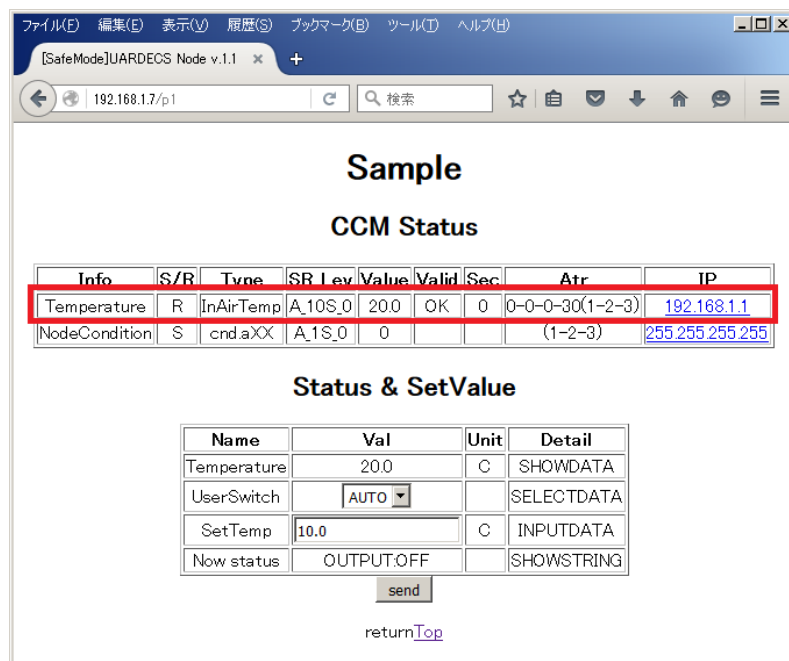
- 9) ここから先では UECS 用パケット送受信ツール (入手先: <https://hoshi-lab.info/uecs.org/arduino/uecsrs.html> または <https://github.com/H-Kurosaki/UecsRS>) を使ってノードの動作を検証します。まず、ツールを入手して ZIP ファイルを解凍し、開発用 PC で実行します。初回起動時に警告が出ることがありますが実行を許可して下さい。



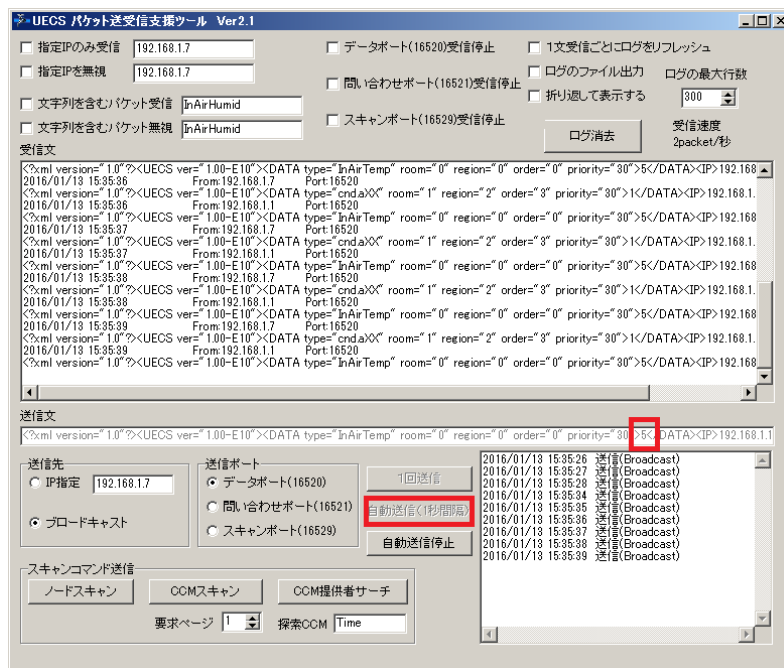
- 10) PC に接続された Thermostat ノードが正常に動作している場合、図のように送信された CCM が一秒間隔で表示されます。



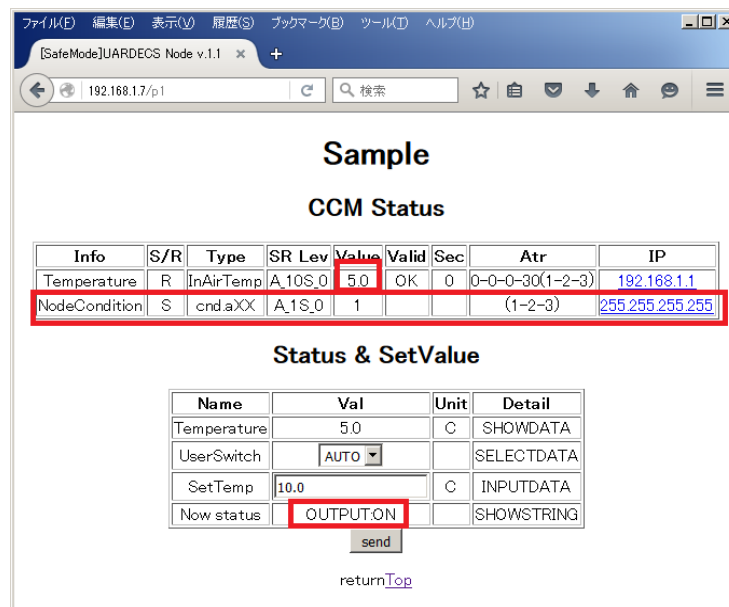
- 1 1) 試しにパケット送受信ツールから InAirTemp の CCM を送信します。起動時の設定のまま自動送信ボタンを押して下さい。



- 1 2) Web ブラウザの画面から Thermostat ノードの Node Status に入ると InAirTemp の項目の Valid が OK に変化し、Value にパケット送受信ツールで送った値が表示されます。これで、PC から送られた CCM を正常に受信していることが分かります。



- 1 3) 送受信ツールで一度パケットの送信を停止した後、InAirTemp の値を 5 に変更してから再度自動送信を行ってみます。



- 1 4) Node Status の InAirTemp の Value が 5.0 に、cnd.aXX の Value が 0 から 1 に変わります。さらに、Now status の所に OUTPUT:ON と表示されます。Thermostat ノードは SetTemp の値(図では 10.0 度)を下回る InAirTemp が入力されると cnd.aXX に 1 を出力するように動作します。

Sample

CCM Status

Info	S/R	Type	SR Lev	Value	Valid	Sec	Atr	IP
Temperature	R	InAirTemp	A_10S_0	5.0	OK	10	0-0-0-30(1-2-3)	192.168.1.1
NodeCondition	S	cmd.aXX	A_1S_0	1			(1-2-3)	255.255.255.255

Status & SetValue

Name	Val	Unit	Detail
Temperature	5.0	C	SHOWDATA
UserSwitch	AUTO		SELECTDATA
SetTemp	10.0	C	INPUTDATA
Now status	OUTPUT:ON		SHOWSTRING

send

[returnTop](#)

Sample

CCM Status

Info	S/R	Type	SR Lev	Value	Valid	Sec	Atr	IP
Temperature	R	InAirTemp	A_10S_0	5.0	-	280	0-0-0-30(1-2-3)	192.168.1.1
NodeCondition	S	cmd.aXX	A_1S_0	0			(1-2-3)	255.255.255.255

Status & SetValue

Name	Val	Unit	Detail
Temperature	5.0	C	SHOWDATA
UserSwitch	AUTO		SELECTDATA
SetTemp	10.0	C	INPUTDATA
Now status	OUTPUT:OFF		SHOWSTRING

send

[returnTop](#)

- 1 5) 送受信ツールからのパケット送信を停止すると InAirTemp の Sec の部分がカウントを始めます(上図)。InAirTemp の送受信レベルは A_10S_0 に設定されていますが、この設定では受信した値の有効期間は 30 秒です。Sec が 30 を超えると Valid が消え、InAirTemp の値が無効になったことを示します(下図)。このように UARDECS は値の有効期間を自動的に管理します。

- 16) 以上で開発環境とマイコンが正常に機能していることが確認できました。もし、コンパイルや通信が正常にできない場合、後述するトラブルシューティングを参照してください。また、近年はAIのアシストにより問題解決ができる場合があります。AIに質問するときは、「使用しているPCとOSを教える」「ソースコードのアップロード」「画面キャプチャ画像のアップロード」「エラーメッセージをペースト」など、こちらから情報を与えたのち質問することで回答の精度が増します。

8 プログラムの構成要素解説

ここでは UARDECS_PICO のコンパイルに最低限必要なソースコードを作るための構成要素を解説する。

1 7) 以下のヘッダファイルを必ず Include する必要がある。

```
#include <Uardec_pico.h>
```

1 8) 必ず初期化が必要なグローバル変数

変数の型	変数名	説明
const byte	U_InitPin	SafeMode ジャンパーピンの番号を指定. このジャンパーをセットして起動された時, IP アドレスが 192.168.1.7, サブネットマスク 255.255.255.0 に強制的に設定される. 購入直後のマイコン, IP アドレスを忘れた時に使用する. ここに設定されたピンは入力モードになり自動的にプルアップされる.
const byte	U_InitPin_Sense	SafeMode ジャンパーピンの判定条件(HIGH または LOW). 起動時に U_InitPin で設定したピンの状態がここに合致するとき, SafeMode となる.
const char PROGMEM	U_name[]	機器の機種名(例えば"Temp controller"). 半角英数で 20 文字以内(タグ, ダブルコーテーション使用禁止). http サーバから出力される html のタイトルと NODESCAN への応答に使用される.
const char PROGMEM	U_vender[]	機器の開発者(例えば"XXX co."). 半角英数で 20 文字以内(タグ, ダブルコーテーション使用禁止). NODESCAN への応答に使用される.
const char PROGMEM	U_uecsid[]	UECS 研究会が発行する ID 番号. 半角英数字 12 文字固定. NODESCAN への応答に使用される. 試作機用の仮設定は"000000000000" ただし自己責任で).

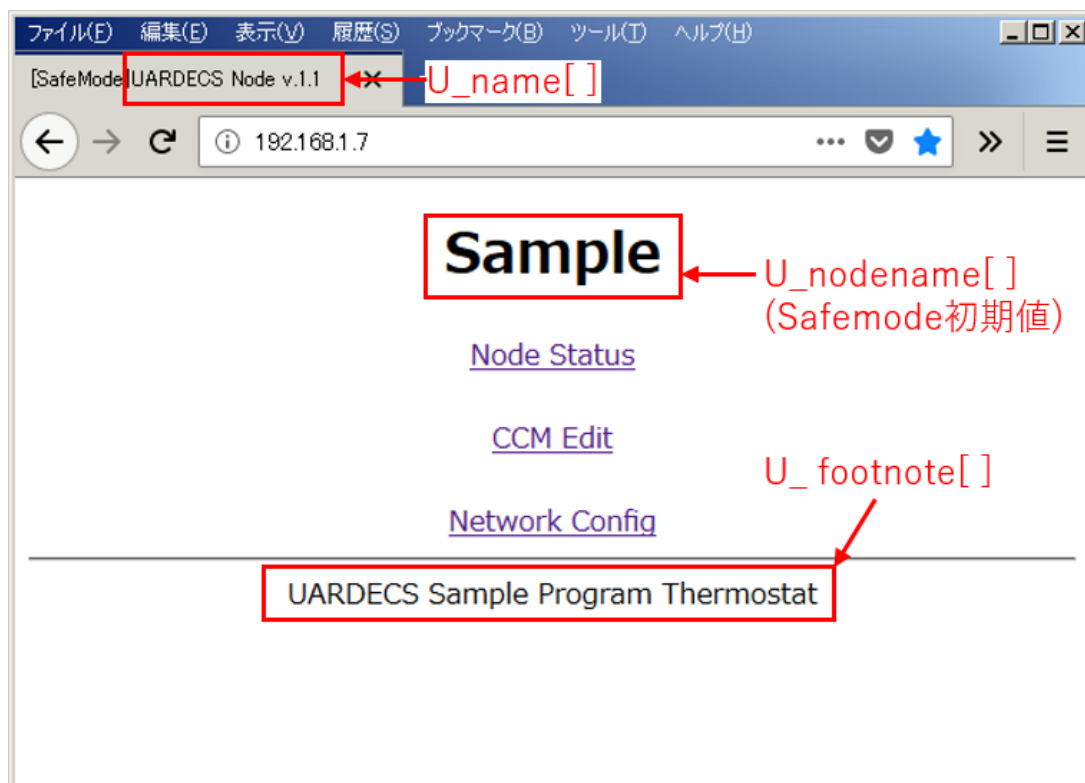
const char PROGMEM	U_footnote[]	ノードの http サーバにアクセスしたときのトップページ の下部に表示する文字列. 字数制限なし日本語使用 可能 . タグはそのまま表示されるので不必要に使うとペ ージのレイアウトが崩れるが, 逆に利用すればリンクな どを張ることもできる.
char 型の配列(数は 20 で固定)	U_nodename[]	ノードの http サーバが表示するノードの名称. 半角英 数字で 19 文字以内, 日本語は UTF-8, 6 文字以内で 使用可能, "&<"使用禁止 . ここで設定された文字列 は SafeMode 時の初期値になる. ユーザーが値を変更 可能で不揮発性メモリにも保存される.
const int	U_MAX_CCM	作成する CCM の総数を整数値で.
const int	U_htmlLine	ノードの http サーバで Node Status 画面に表示する選 択肢など設定項目の総数. 使用しない場合 0 を指定す る.
UECSCCM 構造体の配列	U_ccmList[U_MAX_CCM]	宣言のみで良い.
UECSOriginalAttribute 構 造体	U_orgAttribute	宣言のみで良い.

```

<?xml version="1.0"?><UECS ver="1.00-E10"><NODE>
<NAME>UARDECS Node v.1.1</NAME>
<VENDER>XXXXXXXX Co.</VENDER>
<UECSID>000000000000</UECSID>
<IP>192.168.1.7</IP>
<MAC>112233445566</MAC></NODE></UECS>

```

設定項目の使用箇所①(ノードスキャンへの応答)



設定項目の使用箇所②(Web インターフェースのトップページ)

ファイル(F) 編集(E) 表示(V) 履歴(S) ブックマーク(B) ツール(T) ヘルプ(H)

[SafeMode] UARDECS Node v.1.1 **U_name[]**

192.168.1.7/p1 検索

Sample ← U_nodename[] (SafeMode初期値)

CCM Status

Info	S/R	Type	SR Lev	Value	Valid	Sec	Atr	IP
Temperature	R	InAirTemp	A_10S_0	0.0	-		(253-253-253)	255.255.255.255
NodeCondition	S	cnd.aXX	A_1S_0	0			(253-253-253)	255.255.255.255

U_MAX_CCM

Status & SetValue

Name	Val	Unit	Detail
Temperature	0.0	C	SHOWDATA
UserSwitch	OFF		SELECTDATA
SetTemp	10.0	C	INPUTDATA
Now status	OUTPUT:OFF		SHOWSTRING

U_HtmlLine

send

[returnTop](#)

設定項目の使用箇所③(Web インターフェースの Node Status 画面)

19) 必ず作成が必要な関数

関数名	詳細
void UserInit()	Arduino の起動後、ネットワーク設定を読み込んだ後に呼び出される。この関数の後ネットワークが初期化される。 この関数内では以下の処理を必ず実行する必要がある。 20) mac アドレスの設定、以下のように記述する <pre>U_orgAttribute.mac[0] = 0x12; U_orgAttribute.mac[1] = 0x34; U_orgAttribute.mac[2] = 0x56; U_orgAttribute.mac[3] = 0x78; U_orgAttribute.mac[4] = 0x9A; U_orgAttribute.mac[5] = 0xBC;</pre> [実験用途の場合] 任意の値を設定できますが、同一ネットワーク内の他のノードと MAC アドレスが重複しないようにする。 [製品として販売する場合] 正規に取得した MAC アドレスを記述するか、市販の MAC アドレス ROM を実装し、そこから読み出した値を設定するよう処理を変更する。 21) CCM の作成 [後述する]
void OnWebFormRecieved ()	http サーバの生成した Node Status 画面で Send ボタンが押され、選択肢などの値が送信された時にこの関数が呼び出される。この関数の後、値が不揮発性メモリに格納される。
void UserEverySecond()	この関数は 1 秒毎に呼び出されるので 1 秒間隔で実行すべき処理を記述できる。この関数終了後に 16520 番ポートに送信条件を満たした通信文が送信される。
void UserEveryMinute()	この関数は 60 秒毎に呼び出されるので 60 秒定間隔で実行すべき処理を記述できる。
void UserEveryLoop()	システムのタイマカウント、http サーバの処理、UDP16520,16529,16521 ポートの通信文をチェックした後、呼び出される関数。呼び出される頻度が高いため、重い処理を記述しないこと。
void loop()	この関数内では UECsloop() 関数を呼び出さなくてはならない。必要に応じて処理を記述してもかまわない。呼び出される頻度が高いため、重い処理を記述しないこと。

void setup()	起動後またはリセット後に 1 回だけ呼び出される関数. この関数内では UECSsetup() 関数を呼び出さなくてはならない. 必要に応じて処理を記述してもかまわない. 主にピンの入出力設定, 初期値などを記述する.
--------------	---

2.2) CCM の作成手順

UECS 通信実用規約 1.00-E10 では全てのノードが必ず 1 つ以上の CCM を実装しなければならない. ここでは, CCM の初期化方法を示す.

例としてサンプルプログラムの DummyInAirTemp の一部を示し, 解説する

手順① 作成したい CCM に通し番号を付ける

この部分には整数をそのまま入力しても良いが, 可読性を高めるために DummyInAirTemp スケッチでは以下のように記述している

```
enum {
    CCMID_InAirTemp,
    CCMID_cnd,
    CCMID_dummy,
};
```

このマクロは 2 つの記号定数, CCMID_InAirTemp, CCMID_cnd に通し番号を与えることを示す. CCMID_dummy は作った CCM の総数を数えるのに使うダミーで必ず最後に置く. このマクロによって,

```
InAirTemp      0
CCMID_cnd       1
CCMID_dummy     2
```

のように一意な通し番号が振られる.

以降はこの通し番号を用いて CCM を操作することになる.

手順② CCM の総数を確定し, CCM 格納用の変数を作成する

グローバル変数として以下のように記述する,

```
const int U_MAX_CCM = CCMID_dummy;
UECSCCM U_ccmList[U_MAX_CCM];
```


手順③ CCM を構成する文字列を作成する

必要なのは1つのCCMにつき

1, CCM の説明用文字列(Web 表示用, UTF-8 日本語使用可, 字数制限なし, タグ文字はそのまま出力される)

2, CCM の type 文字列(InAirTemp.mIC など, 半角英字, 半角数字, アンダースコア, ピリオドのみ使用可能, 3-19 文字)この文字列は UARDECS ではそのまま用いられるが, UARDECS_MEGA ではデフォルト値という扱いになり, 後からユーザーが編集した文字列が使われる.

3, CCM の単位を示す文字列(英数のみ 19 文字以下, 不要なときは NULL でも良い)

の3種類である. 全て `const char []` PROGRAMMEM 型のグローバル変数として宣言する.

記述例:

```
// InAirTemp
const char ccmNameTemp[] PROGRAMMEM= "Temperature";
const char ccmTypeTemp[] PROGRAMMEM= "InAirTemp";
const char ccmUnitTemp[] PROGRAMMEM= "C";
// cnd.mIC
const char ccmNameCnd[] PROGRAMMEM= "NodeCondition";
const char ccmTypeCnd[] PROGRAMMEM= "cnd.mIC";
const char ccmUnitCnd[] PROGRAMMEM= ""; //不要なので NULL
```

手順④ UserInit() 関数の中で作成する CCM の数だけ UECSsetCCM() 関数を実行する

UECSsetCCM 関数の要求する値は以下のとおり

```
UECSsetCCM(bool sender,          //true:送信 CCM false:受信 CCM として宣言する
            int num,              //手順①で付けた CCM の通し番号
            char* name,           //手順③の CCM の説明用文字列のポインタを与える
            char* type,           //手順③の CCM の type 文字列のポインタを与える
            char* unit,           //手順③の CCM の単位文字列のポインタを与える
            unsigned short priority, //通常は整数値 29, UECS 通信実用規約 1.00-E10 参照
            unsigned char decimal, //値に小数を使う場合の小数点桁数, 0 で整数を示す
            signed char ccmlevel  //送受信の頻度[6) を参照]
);
```

※UECSsetCCM 関数は UserInit() 内での使用を推奨する. UserInit() 外での利用は不具合の原因になるので, そのような実装は行わないこと.

記述例：

```
void UserInit()
{
UECSsetCCM(true, CCMID_InAirTemp, ccmNameTemp, ccmTypeTemp, ccmUnitTemp, 29, 1, A_1S_0);
UECSsetCCM(true, CCMID_cnd, ccmNameCnd, ccmTypeCnd, ccmUnitCnd, 29, 0, A_10S_0);
. . . .
```

2 3) CCM の送受信の頻度（レベル）について

UECS 通信実用規約“1.00-E10”で定める送受信頻度の実装状況は以下のようになっている

送受信 レベル	送信頻度	受信有効期限	UARDECS_PICO での送 信	UARDECS_PICO での 受信
A_1S_0	1 秒	3 秒	使用可能	使用可能
A_1S_1	1 秒 値が変化した時も送信	3 秒	A_1S_0 と同じ動作	A_1S_0 と同じ動作
A_10S_0	10 秒	30 秒	使用可能	使用可能
A_10S_1	10 秒 値が変化した時も送信	30 秒	使用可能※1	使用可能
A_1M_0	60 秒	180 秒	使用可能	使用可能
A_1M_1	60 秒 値が変化した時も送信	180 秒	使用可能※1	使用可能
B_0	送信要求があった時	定義しない	未実装	未実装
B_1	送信要求があった時 値が変化した時も送信	定義しない	未実装	未実装
S_1S_0	遠隔操作中:1 秒 遠隔操作しない時:停止	3 秒	A_1S_0 と同じ動作 送信停止は手動実装	使用可能
S_1M_0	遠隔操作中:60 秒 遠隔操作しない時:停止	180 秒	A_1M_0 と同じ動作 送信停止は手動実装	使用可能

※1 値の変化を検出した場合，1 秒以内に送信される．ただし，値の変化を検出する頻度は 1 秒間隔なので，それ以上高頻度のパケット送信は行われない．

2 4) CCM の送信

UECSsetCCM() 関数で sender を true とし、送信 CCM として登録したものは、設定した送信頻度に従い、一定時間間隔で自動送信される。CCM に値を書き込むときは、以下のように記述する。

記述例：

```
void UserEverySecond()
{
    U_ccmList[CCMID_InAirTemp].value=123;
}
```

value は long 型の整数である。

UECSsetCCM() 関数の decimal 値を 1 以上に設定した場合、value は固定小数点数となり、下位の桁は小数点下の値を表す。例えば上記の記述例において decimal を 1 で宣言した時、CCM として送出される値は 12.3 となる。decimal を 2 で宣言した時は 1.23 となる。

2 5) CCM の受信

UECSsetCCM() 関数で sender を false とし、受信 CCM として登録したものは、設定した受信頻度に従い、値の有効期限が管理される。値が有効かどうかは以下の方法で検出できる。

記述例：

```
if(U_ccmList[CCMID_InAirTemp].validity==true)
{
    //値は有効
}
else
{
    //値は無効
}
```

必ず値の有効/無効を確認してから処理することを勧める。もし、UECSsetCCM() 関数で宣言時の decimal が 0 であれば、そのまま long 型の変数として扱える。

記述例：

```
long ccmTemp= U_ccmList[CCMID_InAirTemp].value;
```

宣言時の decimal が 1 の場合、プログラム内で浮動小数点型に変換するには以下のようにキャストする必要があるかもしれない。

記述例：

```
float ccmTemp=(float)U_ccmList[CCMID_InAirTemp].value/10;
```

※受信 CCM の固定小数点の仕様について

例えば decimal が 2 の状態で外部から以下の CCM 値を受信した場合は次のようになる

受信値	U_ccmList[CCMID].value の値
100.1	10010
100.12	10012
100.123	10012
100	9999 (変換誤差が発生する)
10	999 (変換誤差が発生する)

2.6) CCM を送信停止する方法

送信頻度に S_1S_0などを指定した場合、CCM の送信を一時停止する必要がある。このようなときは、UserEverySecond() 関数内に以下のように記述する

```
U_ccmList[CCMID].flagStimeRfirst=false;
```

flagStimeRfirst の値は、この関数を抜けると上書きされるので、flagStimeRfirst に false を書き込んでいる間だけ送信が停止し、この処理を止めると自動的に送信が再開される。この方法を UserEverySecond() 関数外で利用することはできない。

2 7) U_ccmList[] 構造体の詳細仕様
(UARDECS と UARDECS_MEGA に共通の仕様)

UECSCCM 構造体の主要メンバー	変数の型	送信時	受信時
sender	boolean	TRUE	FALSE
name	const char * PROGMEM	文字列ポインタ CCM の説明用文字列 (Web 表示用, UTF-8 日本語使用可, 字数制限なし, タグ文字はそのまま出力される)	
type	const char * PROGMEM	文字列ポインタ CCM の type 文字列 (InAirTemp.mIC など, 半角英字, 半角数字, アンダースコア, ピリオドのみ使用可能, 3-19 文字) ユーザーが初期化処理する時に利用される.	
unit	const char * PROGMEM	文字列ポインタ CCM の単位を示す文字列 (英数のみ 19 文字以下, 不要なときは NULL でも良い)	
ccmLevel	char	記号定数で記述された UECs の通信規約で定められた送受信レベル.	
value	signed long	送信値を格納する	受信値が格納される
decimal	unsigned char	value の小数点以下の桁数	
validity	boolean	定期送信 CCM では送信実績のある場合 true に, 規定の送信頻度を満たさない場合 false になる.	登録した CCM が時間内に取得できている場合 true. タイムアウトで false になる.
recmillis	signed long	UARDECS では未使用. UARDECS_MEGA の定期送信 CCM では前回送信後の経過時間を ms 単	最後に受信してからの経過時間を ms 単位で示す. 最大 24 時間分カウントされる (誤差

		位で示す. 最大 10 時間までカウント. 送信レベル B_?? , S_?? の CCM では値は保証されない.	あり). Web 上に表示されるのは 10 時間まで. flagStimeRfirst が true でない場合, 値は保証されない.
address	IPAddress	未使用	CCM 送信元の IP アドレス
flagStimeRfirst	boolean	CCM 送信直前に true になる. false にセットすると送信をキャンセルする. (UserEverySecond() 関数内でのみ操作可能)	初めて CCM 受信に成功すると true にセットされる.
attribute[4]	signed short	attribute[3] の priority のみ利用.	受信した CCM に記述された属性値 (baseAttribute と異なることもある) attribute[0] は room attribute[1] は region attribute[2] は order attribute[3] は priority に対応.
baseAttribute[4]	signed short	Web の Network Config 画面で設定したノードの基本属性 baseAttribute[0] は room baseAttribute[1] は region baseAttribute[2] は order	
typeStr[20]	char	UARDECS_MEGA ではユーザーが編集した Type 文字列がこの変数に記録されており CCM 送信時に利用される. Web 上から書き換えられる時に同じ文字列が不揮発性メモリ上にも複写され, 電源投入時に読み出される.	

old_value	signed long	変化があったら送信する CCM で利用される. 過去に送信済みの値を記録する.	未使用
-----------	-------------	--	-----

送信 CCM の属性値は

U_ccmList[CCMID].baseAttribute[0] →room

U_ccmList[CCMID].baseAttribute[1] →region

U_ccmList[CCMID].baseAttribute[2] →order

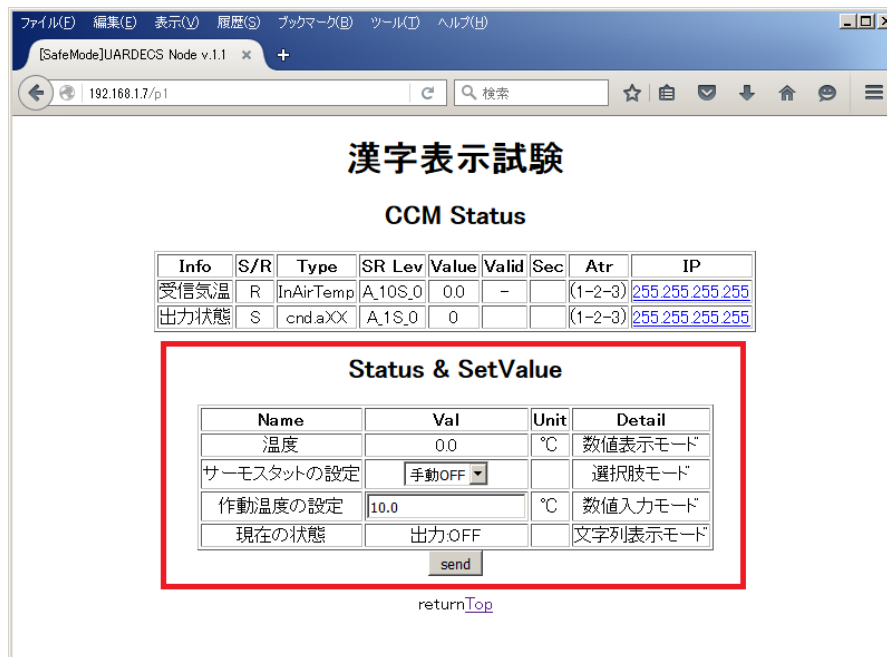
U_ccmList[CCMID].baseAttribute[3]→priority

U_ccmList[CCMID].value→value(decimal により小数点の値を設定)

として出力される.

受信 CCM の場合, UECS 通信実用規約"1.00-E10"で定めた方法で CCM の仕分けを行い, 有効と見なされたパケットのみが解釈されてその属性値と送信元の IP アドレスが代入される.

1) Web インターフェースについて



Web インターフェースはノードにブラウザでアクセスした時、Node Status 画面の下部に表示される設定項目のことである。この設定値は不揮発性メモリに記録され、電源を切っても起動時に自動復帰する。ただし、SafeMode では不揮発性メモリからの値の読み出しは行われず、別途指定した初期値が設定される(不揮発性メモリへの書き込みは可能)。Web インターフェースは以下の4種類の入出力カラムを表示できる。

①UECSSHOWDATA 数値表示カラム(出力用)

指定した数値(整数または固定小数点数)を表示する。

②UECSSHOWSTRING 文字列表示カラム(出力用)

あらかじめ登録した文字列の中から指定した1つの文字列を表示する。

③UECSINPUTDATA 数値入力欄の表示(入力用)

数値入力専用の欄を表示する。整数または固定小数点数の入力が可能である。

④UECSSELECTDATA 選択肢の表示(入力用)

選択肢(ドロップダウンリスト)を表示する。

2) Web インターフェースの作成手順

手順①

作成する入出力カラムの総数をグローバル変数 HtmlLine に記述する.

記述例 :

```
const int U_HtmlLine = 1;
```

手順②

カラムからの入出力用に long 型変数を 1 つのカラムにつき 1 つグローバル変数として作成する.

記述例 :

```
long webValue;
```

複数のカラムでこの変数を共用した場合, 正常な動作は保証できない.

手順③

カラムに表示する素材のうち, カラム名, 単位, 詳細説明の文字列を準備する. 全て const char [] PROGMEM 型のグローバル変数として宣言する.

記述例 :

```
const char NAME[] PROGMEM= "Temperature";  
const char UNIT[] PROGMEM= "C";  
const char NOTE[] PROGMEM= "SHOWDATA";
```

この文字列は UTF-8 で日本語使用可能であり, 字数制限は無い. タグはそのまま表示されるので注意すること.

手順④-1 UECSHOWDATA 数値表示カラム(出力用)の場合

UECSUserHtml 構造体をグローバルとして作成し，素材を入力する．入力する順番は以下のようになる．ダミー値は全て未使用である．UECSHOWDATA は表示値用変数に格納された値を表示する．小数桁数に 1 以上が指定された場合，固定小数点数として扱う．例えば表示値用変数が 123 で小数桁数が 1 の場合，表示は 12.3 となる．

記述例：

```
const char** DUMMY = NULL; //(const char** 型のダミー値を準備)
struct UECSUserHtml U_html[U_HtmlLine]={
    NAME,           //カラム名(const char* PROGMEM)
    UECSHOWDATA,    //カラムの種類(記号定数)
    UNIT,           //単位(const char* PROGMEM)
    NOTE,           //詳細説明(const char* PROGMEM)
    DUMMY,          //ダミー値(const char**)
    0,              //ダミー値(unsigned char)
    &(webValue),    //表示値用変数(long *)
    0,              //初期化値(long) SafeMode で webValue の初期値になる
    0,              //ダミー値(long)
    1               //小数桁数(unsigned char)
};
```

手順④-2 UECSSHOWSTRING 文字列表示コラム(出力用)の場合

あらかじめ、素材文字列を準備する必要がある。全て `const char []` PROGMEM 型のグローバル変数として宣言する。この文字列は UTF-8 で日本語使用可能であり、字数制限は無い。タグはそのまま表示されるので注意すること。初回起動時に異常な文字列が出力される場合は、SAFEMODE で起動し Node Status 画面で送信ボタンを 1 回押すと修正される。

記述例：

```
const char SHOWSTRING_OFF[] PROGMEM= "OUTPUT:OFF";
const char SHOWSTRING_ON [] PROGMEM= "OUTPUT:ON";
```

次に、素材文字列のポインタを列挙した配列をグローバル変数として準備する。

記述例：

```
const char *stringSHOW[2]={
    SHOWSTRING_OFF,
    SHOWSTRING_ON,
};
```

次に UECSUserHtml 構造体をグローバルとして作成し、素材を入力する。入力する順番は以下のようになる。ダミー値は全て未使用である。

記述例：

```
struct UECSUserHtml U_html[U_HtmlLine]={
    {
        NAME,                //コラム名(const char* PROGMEM)
        UECSSHOWSTRING,      //コラムの種類(記号定数)
        UNIT,                //単位(const char* PROGMEM)
        NOTE,                //詳細説明(const char* PROGMEM)
        stringSHOW,          //素材文字列のポインタ列挙配列(const char**)
        2,                   //素材文字列の総数(unsigned char)
        &(webValue),          //表示する素材文字列の通し番号 (long *)
        0,                   //初期化値(long) SafeMode で webValue の初期値になる
        0,                   //ダミー値(long)
        0                    //ダミー値(unsigned char)
    }
};
```

上記例では `webValue =0` の時、Web 上には“OUTPUT:OFF”が表示され、`webValue =1` の時、“OUTPUT:ON”が表示される。

手順④-3 UECSINPUTDATA 数値入力欄の表示(入力用)の場合

UECSUserHtml 構造体をグローバルとして作成し、素材を入力する。入力する順番は以下のようになる。ダミー値は全て未使用である。

記述例：

```
const char** DUMMY = NULL; //(const char** 型のダミー値を準備)
struct UECSUserHtml U_html[U_HtmlLine]={
    NAME,          //カラム名(const char* PROGMEM)
    UECSINPUTDATA, //カラムの種類(記号定数)
    UNIT,          //単位(const char* PROGMEM)
    NOTE,          //詳細説明(const char* PROGMEM)
    DUMMY,         //ダミー値(const char**)
    0,             //ダミー値(unsigned char)
    &(webValue),   //入力値格納変数(long *)
    -1000,         //最小値(long) SafeMode で webValue の初期値になる
    1000,          //最大値(long)
    1              //小数桁数(unsigned char)
};
```

UECSINPUTDATA は数値入力フォームを生成し、入力された値を入力値格納変数に格納する。ノードの http サーバが値を受信すると、OnWebFormRecieved() 関数が実行され、この中で入力値格納変数を参照することで更新された値を受け取ることができる。

OnWebFormRecieved() 関数実行の後、入力値格納変数の値は自動的に不揮発性メモリに保存される。したがって、受信した値に何らかの操作を加えてから不揮発性メモリに保存させることも可能である。ノードへの電源投入時 UECSsetup() 関数実行後に webValue の値は自動復帰する(SafeMode 以外)。小数桁数に 1 以上が指定された場合、入力値は固定小数点数として扱う。最小値、最大値は入力値がこの範囲外にあるとき、自動的に入力値をこの範囲内にクリップする(必ず設定する必要がある)。

例えば小数桁数が 2 の状態で UECSINPUTDATA カラムへの入力値は以下のようになる

入力値	入力値格納変数の値
100.1	10010
100.12	10012
100.123	10012
100	9999 (変換誤差が発生する)
10	999 (変換誤差が発生する)

手順④-4 UECSSELECTDATA 選択肢の表示(入力用)の場合

あらかじめ、選択肢文字列を準備する必要がある。全て `const char [] PROGMEM` 型のグローバル変数として宣言する。この文字列は UTF-8 で日本語使用可能であり、字数制限は無い。タグはそのまま表示されるので注意すること。

記述例：

```
const char SELECT0[] PROGMEM= "選択肢 0";
const char SELECT1[] PROGMEM= "選択肢 1";
const char SELECT2[] PROGMEM= "選択肢 2";
const char SELECT3[] PROGMEM= "選択肢 3";
```

次に、選択肢文字列のポインタを列挙した配列をグローバル変数として準備する。

記述例：

```
const char *stringSELECT[4]={
SELECT0,
SELECT1,
SELECT2,
SELECT3,
};
```

次に `UECSUserHtml` 構造体をグローバルとして作成し、素材を入力する。入力する順番は以下のようになる。ダミー値は全て未使用である。

記述例：

```
struct UECSUserHtml U_html[U_HtmlLine]={
{
NAME,                //カラム名(const char* PROGMEM)
UECSSELECTDATA,     //カラムの種類(記号定数)
UNIT,                //単位(const char* PROGMEM)
NOTE,                //詳細説明(const char* PROGMEM)
stringSELECT,        //選択肢文字列のポインタ列挙配列(const char**)
4,                  //選択肢文字列の総数(unsigned char)
&(webValue),         //入力値格納変数(long *) 選ばれた選択肢の番号
0,                  //初期化値(long) SafeMode で webValue の初期値になる
0,                  //ダミー値(long)
0                    //ダミー値(unsigned char)
}};
```

ノードの http サーバが値を受信すると、OnWebFormRecieved()関数が実行され、この中で入力値格納変数を参照することで更新された値を受け取ることができる。上記例では webValue =0 の時、“選択肢 0”が選ばれている事を示し、webValue =3 の時“選択肢 3”が選ばれている事を示す。OnWebFormRecieved()関数実行の後、入力値格納変数の値は自動的に不揮発性メモリに保存される。したがって、受信した値に何らかの操作を加えてから不揮発性メモリに保存させることも可能である。ノードへの電源投入時 UECSsetup() 関数実行後に webValue の値は自動復帰する(SafeMode 以外)。

3) Web インターフェースについて補足事項

①Web インターフェースが不要なとき

Web インターフェースは CCM 関連の機能とは完全に独立しており、不要な場合は表示しないこともできる。この場合、グローバル変数の初期化部分に以下の 2 行を記述するだけでよい。

記述例：

```
const int U_HtmlLine = 0;
struct UECSUserHtml U_html[U_HtmlLine]={};
```

②文字列素材の使い回し

Web インターフェース用に準備した文字列は複数のカラムで使い回すことでフラッシュメモリの消費量を節約できる。

③作れる入出力カラムの最大数

UARDECS_PICO では web サーバに入力される文字列が 499byte を超えた場合、末端を処理できない。URL に付加される文字列は、入力された数値の桁数などで大きく変動するため一概に入出力カラムの最大数を定めることはできないが、32bit の long 型変数が表現できる整数の最大値が 10 桁で、これに符号、少数点、セパレータ文字等を加えると、1 カラム辺り最大 15 byte を消費する。さらに、http サーバへのコマンドや末端のフッタが約 14byte を消費する。したがって、32 個以内に留めたほうが無難である。

④Web インターフェースに設定された値の書き換え

プログラム上から選択肢の位置や入力値の格納変数の内容を書き換える場合、通常は OnWebFormRecieved() 内でのみ編集が可能である。それ以外の場所で単純に変数の値だけ書き換えてもフラッシュメモリに書き込まれないので正常に反映されない。直接書き換えたい場合はフラッシュメモリの値と整合性を取るため、以下の ChangeWebValue() 関数を使用する。

```
bool ChangeWebValue(  
    signed long* data,      //書き換えたい変数のアドレス  
    signed long value      //書き込む値  
);
```

戻り値：書き込み成功:true 書き込み失敗:false

記述例：

```
ChangeWebValue(&(webValue), 123);
```

&(webValue)のところには Web インターフェース用に登録された変数のアドレスを記述する。それ以外の変数を記述すると書き込み失敗になる。

この関数は Web インターフェースに設定された最大値、最小値を無視して強制的に値を書き込むので注意すること。また、フラッシュメモリには書き換え回数に寿命があるので高頻度の書き換えは推奨しない。特に細かい単位での書き換えが可能だった Arduino と異なり Raspberry Pi Pico のフラッシュメモリは一か所書き換えるだけでも領域全体の寿命を損耗するので書き換え回数では不利である。SafeMode では、この関数を用いて値を書き換えても特定の操作時に値が初期化されることがある。

10 補足事項と Arduino からの移行時の注意

4) 16521 ポートの応答について

16521 ポートの通信内容は、CCM のサーチと CCM の送信要求ですが、定期送信 CCM(レベル A_??のもの)では、送信実績がある場合しか応答を返しません。これは、故障して CCM 送信を停止しているセンサなどを誤って参照しないための措置です。そのため、ノードの電源を入れてから応答するようになるまで時間がかかることがあります。CCM の送信要求は B_??レベルの CCM に対応していないため、実装されていません。

5) CCM 送信タイミングの分散について

起動タイミングが同期すると複数のノードから特定の秒に大量の CCM が送信されてしまうため、パケットロスの原因になっていました。そのため定期送信 CCM のうち、A_10S_?と A_1M_?に限り、送信のタイミングを分散します。UARDECS_PICO では 10-60 秒の範囲で分散して送信します。設定されている IP アドレスの下位の数値により送信タイミングにばらつきを与えます。ただし、1 秒間隔で送信される CCM に対して送信タイミングの分散は適用されません。

6) Arduino 版とのフラッシュメモリの書き込み耐性の違いについて

本ライブラリは Arduino に搭載されていた EEPROM の代用として Raspberry Pi Pico のプログラムフラッシュの未使用領域 4096byte を保存領域に使用します。また、SRAM を 4096byte 分キャッシュ領域として使用する(起動後に確保されるので注意)ので、この分だけ利用可能なメモリが減少します。EEPROM の耐久性は書き換え回数のみは Arduino と同等(100,000 回)ですが、4096byte 単位でしか書き換えができないため、一部書き換えただけでも 4096byte の領域全体の耐久性を損耗します。これらは書き換え頻度が少ない IP アドレスの保存やその他設定データの保存には深刻な影響はないかもしれませんが、データのロギングなど一定時間間隔で書き込むような用途には著しく不利になります。(おそらく)byte 単位で書き換えが可能な Arduino より遥かに少ない回数で限界に達します。プログラム中で EEPROM を高頻度に書き換える場合、別途不揮発性のデータが保存可能なデバイスを接続する設計にすることを推奨します。

7) char 型変数の仕様の差異について

Arduino では char 型変数は標準で符号付き(signed char)と解釈されますが、Raspberry Pi Pico 系の CPU では符号なし(unsigned char)と解釈されます。そのため、同じソースコードをコンパイルしたとき、演算に差が出る場合があります。char 型変数を符号付きとして演算に用いる場合は signed を明示する必要があります。

1 1 よくあるトラブルと対処法

- 1) Web からの設定が有効にならない, 設定した値がリセットすると元に戻っている

SafeMode 中では起動時に Web 上の設定にソースコード内に記述したデフォルト値が設定されます(工場出荷時のフラッシュメモリ上にはプログラムの許容外の値が書き込まれていることがあり, これを読み込んでノードが異常動作を起こすのを防ぐため). SafeMode を抜けて下さい.

- 2) ” Compilation error: exit status 0xc0000409” と表示されてサンプルスケッチがコンパイルできない

Arduino IDE 内蔵の Python とすでにインストールされている Python が衝突している可能性があります. 以下の処理で解決することがあります.

- ・ “C:\Users\¥<Windows ユーザー名>\AppData\Local\Arduino15\packages\¥rp2040¥tools\¥pqt-python3¥” のフォルダを丸ごと削除
- ・ Arduino IDE を起動し, Arduino IDE → ボードマネージャ → 「Raspberry Pi Pico /RP2040 by Earle F. Philhower, III」を一度アンインストール
- ・ IDE を再起動 → 「Raspberry Pi Pico/RP2040 by Earle F. Philhower, III」を再インストール

- 3) サンプルスケッチをコンパイルして転送しても CCM が送信されない

- ①本当に「書き込みが完了しました」と表示されている?

プログラムの転送に失敗する場合, (1)文法が間違っておりコンパイルできない (2)マイコンの種類が間違っている, 特に Pico と Pico2 の選択を間違えると, プログラムの転送が完了しても実行されません. (3)COM ポートの指定が間違っている (4)以前にコンパイルした時の処理が終わっていない, 等の問題があることが多いです.

- ②電源容量が足りない

LAN 接続中の W5500 は最低でも約 170mA を消費し, さらに接続したデバイスの消費電力が上乘せされます. 電源容量が足りないと起動時に電源がダウンすることがあります. イーサネットコントローラーは 3.3V の電源を多く使用する傾向があります.

③CCM は送信されているが PC のファイアウォールでブロックされている
特に企業向けのセキュリティソフトは UECS で使用するポート (UDP16520, UDP16521, UDP16529) をブロックしてしまうことがあります。この状況だと、PC 側でパケットモニタ用のツールを動作させても何も表示されません。

4) ノードから PC に CCM は到達しているが、Web サーバにアクセスできない

①ノードと PC のサブネットマスクが食い違っている、IP アドレスが不適切
例えばノードと PC のサブネットマスクが 255. 255. 255. 0 の場合、それぞれに 192. 168. 1. 1～192. 168. 1. 254 の範囲で重複しない IP アドレスを付ける必要があります。

②MAC アドレスが重複している

MAC アドレスが同じノードが複数あると、http の応答が正常にできません。必ず MAC アドレスは全てのノードに異なる値を設定して下さい。

③少し待ってみる

起動から Web ページにアクセス可能になるのに 10 秒ぐらいかかる場合もあります。
(特にスイッチング Ethernet HUB を使用していると MAC アドレスが登録されるのに時間がかかります)

5) ノードから PC に CCM は到達しているがノードスキャン、CCM スキャンに応答がない
ノードスキャン、CCM スキャンはブロードキャストではなく 1 対 1 通信のユニキャストになるので、4) と同じ項目を点検して下さい。

6) UECS 用パケット送受信ツールでノードにパケットを送っているが反応がない

① CCM の文法が合っていない

CCM の先頭に不正な文字列がある (末端のタグ外の文字列は無視されます)、ヘッダが一致しない、room-region-order-priority の順番が違う、IP タグが足りない等で弾かれることがあります。

②PC に複数の LAN ポートがある

有線 LAN+無線 LAN など複数の LAN ポートがある場合、UECS 用パケット送受信ツールで CCM をブロードキャストできるのはどれか 1 つのポートだけになります。不要な LAN ポートを無効化するか、IP 指定送信を行って下さい。

7) Web ページのタイトルに[ERR]と表示される

メモリーリークしています。特に文字列の字数制限が守られていない時に発生しますが、正規の利用方法で発生する場合、状況などを開発者に連絡いただけると幸いです。

8) CCM のサーチに応答しない

UARDECS_PICO ではサーチコマンドへの応答も実装されています。ただし、定期送信 CCM では登録があっても送信実績のある CCM しか応答しません。そのため電源を入れてから応答するようになるまで時間がかかることがあります。

9) WDT を実装したが、回線が不安定な環境下で http アクセスすると WDT が作動する

内蔵 web サーバでのデータ送信中に回線が切断されると W5500 のデフォルト設定ではタイムアウトまで少なくとも 32 秒のロック時間が発生します。WDT を使用する場合、このロック時間をフリーズと誤認して WDT が作動してしまうことがあります。この問題を解決するにはイーサネットドライバを改変し、タイムアウト待ちのループ中に WDT をリセットする処理を追加する必要があります。

1 2 更新履歴

更新履歴は GitHub リポジトリを参照してください

https://github.com/H-Kurosaki/UARDECS_PICO